

การใช้ mongoDB

ในการทำฐานข้อมูล



เนื้อหาที่จะเรียน

2

- รู้จักกับฐานข้อมูล
- รู้จัก MongoDB
- การใช้งาน MongoDB
- โครงสร้างฐานข้อมูล
- ชนิดข้อมูลและการดำเนินการกับข้อมูล
- การสร้าง Index และการสืบค้นข้อมูล



Database หรือ ฐานข้อมูล

- กลุ่มของข้อมูลที่ถูกเก็บรวบรวมไว้ ทำให้มีการเข้าถึงข้อมูลง่ายและสะดวกต่อการนำไปใช้งาน
- ปัจจุบันฐานข้อมูลที่นิยมใช้งานแบ่งออก 2 แบบ
 - ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)
 - ฐานข้อมูลไม่ใช่เชิงสัมพันธ์ (Non-Relational Database)



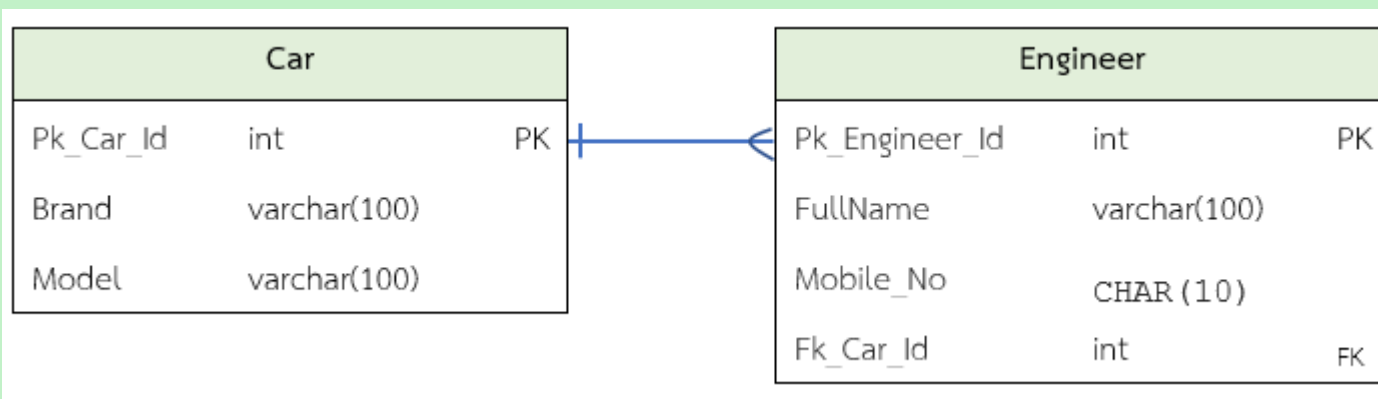
ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

- SQL Databases หรือที่เรียกกันว่า “ฐานข้อมูลแบบเชิงสัมพันธ์”
- ระบบที่รวบรวมข้อมูลต่าง ๆ ที่เกี่ยวข้องกันเข้าไว้ด้วยกันอย่างมีระบบมีความสัมพันธ์ระหว่างข้อมูลต่าง ๆ ที่ชัดเจน
- ใช้ภาษา SQL (Structure Query Language) ในการดำเนินการกับข้อมูลไม่ว่าจะเป็นการเพิ่ม ลบ แก้ไข และการค้นหาข้อมูล
- จัดเก็บข้อมูลแบบตาราง (Table) ซึ่งในแต่ละตารางจะประกอบไปด้วย Row และ Column
- มีการกำหนดโครงสร้างในการจัดเก็บข้อมูลที่ชัดเจน (Structure Data)
- มีการเชื่อมโยงความสัมพันธ์กับข้อมูลตารางอื่น ๆ โดยใช้ Key เพื่อกำหนดความสัมพันธ์ระหว่างตารางทำให้ง่ายต่อการเข้าถึงข้อมูลของกันและกัน



ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

5



Car

Pk_Car_Id	Brand	Model
1	Toyota	2023 Camry Hybrid
2	Mazda	CX-30

1

Engineer

Pk_Engineer_Id	FullName	Mobile_No	Fk_Car_Id
1	Benjamin K	0805563554	1
2	Oliver L	0991564235	2
3	Emma W	0406655844	2
4	Lucas L	0675233975	1

M



ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

เมื่อมีปริมาณการจัดเก็บข้อมูลมากขึ้น ทำให้ต้องขยายพื้นที่ในการจัดเก็บข้อมูลหรือปรับขนาดฐานข้อมูลเพื่อรองรับข้อมูลมากขึ้นฐานข้อมูลเชิงสัมพันธ์จะปรับขนาดในแนวตั้ง (Vertical-Scaling) โดยติดตั้งอุปกรณ์เพื่อเพิ่มพื้นที่ในการเก็บข้อมูลเพิ่มแรมเพิ่มหน่วยประมวลผล ต้องใช้ค่าใช้จ่ายสูงและจะต้องปิดระบบเพื่อปรับขนาดฐานข้อมูล

ตัวอย่างของ SQL Databases ได้แก่ MySQL, Oracle, และ PostgreSQL เป็นต้น



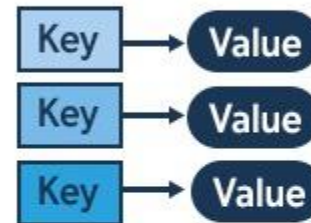
ฐานข้อมูลไม่ใช่เชิงสัมพันธ์ (Non-Relational Database)

7

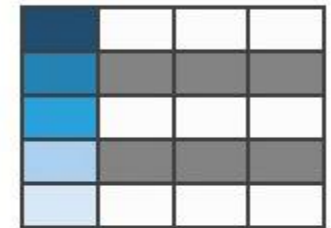
- NoSQL databases หรือที่เรียกกันว่า “ฐานข้อมูลไม่ใช่แบบเชิงสัมพันธ์” หรือเรียกว่า Not only SQL
- มีวิธีการจัดเก็บข้อมูลที่ยืดหยุ่น สามารถปรับเปลี่ยนโครงสร้างในการจัดเก็บข้อมูลได้
- เมื่อมีปริมาณการจัดเก็บข้อมูลมากขึ้น ทำให้ต้องขยายพื้นที่ในการจัดเก็บข้อมูลหรือปรับขนาดฐานข้อมูลเพื่อรองรับข้อมูลที่มากขึ้น ฐานข้อมูลไม่ใช่เชิงสัมพันธ์จะปรับขนาดในแนวนอน (Horizontal-Scaling) โดยเพิ่มเซิร์ฟเวอร์ สิทธิภาพสูง เน้นการง่าย ไม่ต้องปิดระบบเพื่อปรับขนาดฐานข้อมูล

NoSQL

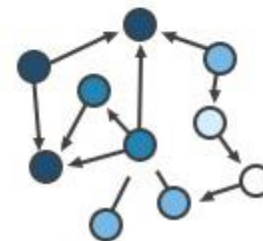
Key-Value



Column-Family



Graph



Document



<https://www.geeksforgeeks.org/types-of-nosql-databases/>



รูปแบบการจัดเก็บข้อมูล NoSQL

- แบ่งออกเป็น 4 ประเภทคือ
- 1. Key-Value Store** เป็นการเก็บข้อมูลที่ไม่ซับซ้อน เก็บข้อมูลในรูปแบบ key:value หรือเรียกว่า Dictionary ทำให้สามารถเข้าถึงข้อมูลได้รวดเร็ว จะใช้ Key ทำหน้าที่เป็นตัวเชื่อมโยงไปยังค่า Value ดังนั้นการดึงข้อมูลจะต้องระบุ key ที่เก็บ value นั้น ๆ
- ตัวอย่าง Database Software ที่ใช้งานลักษณะนี้ ได้แก่ Redis, Memcached, Apache Ignite, Couchbase, Dynamo

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

<https://blog.cloudhm.co.th/sql-vs-nosql/>



รูปแบบการเก็บข้อมูล NoSQL (ต่อ)

- แบ่งออกเป็น 4 ประเภทคือ
- 2. Document Store** เป็นการเก็บข้อมูลเชิงเอกสาร ที่เรียกว่า “Document-Oriented” เป็นการเก็บข้อมูลประเภทกึ่งโครงสร้าง (Semi-structure data) เช่น JSON หรือ XML โดยใช้โครงสร้างภายในเอกสารเพื่อระบุการจัดเก็บ ดึง และจัดการข้อมูล
- ตัวอย่าง Database Software ที่ใช้งานลักษณะนี้ ได้แก่ Cosmos DB, IBM Domino, MongoDB, Couchbase, ArangoDB

ID	Name	<u>IsActive</u>	Dob
1	John Smith	True	8/30/1964
2	Sarah Jones	False	2/18/2002
3	Adam Stark	True	7/13/1987

Relational Databases

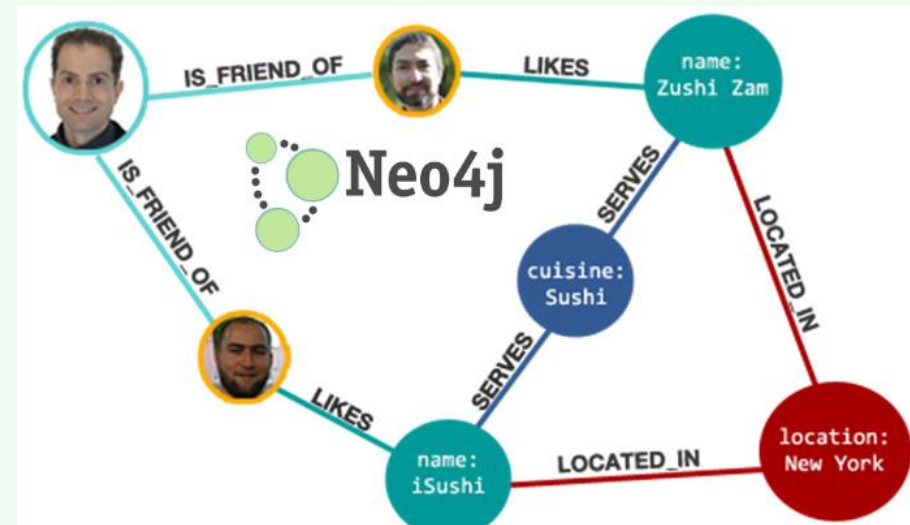
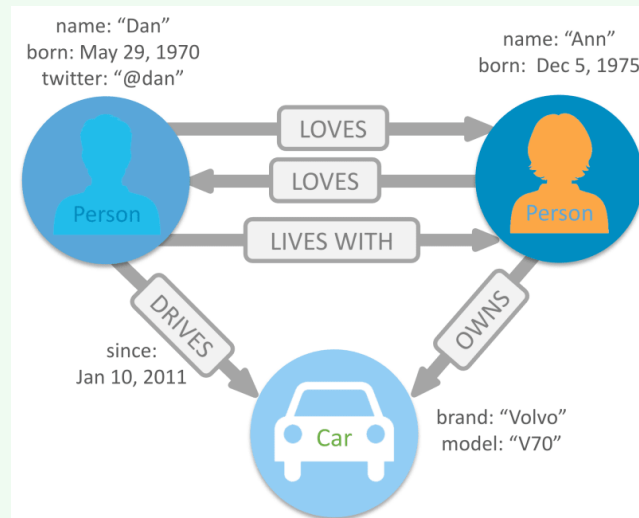
Document 1	Document 2	Document 3
<pre>{ "id": "1", "name": "John Smith", "isActive": true, "dob": "1964-30-08" }</pre>	<pre>{ "id": "2", "fullName": "Sarah Jones", "isActive": false, "dob": "2002-02-18" }</pre>	<pre>{ "id": "3", "fullName": { "first": "Adam", "last": "Stark" }, "isActive": true, "dob": "2015-04-19" }</pre>

NoSQL Document Databases



รูปแบบการจัดเก็บข้อมูล NoSQL (ต่อ)

- แบ่งออกเป็น 4 ประเภทคือ
- **3. Graph Store** จัดเก็บข้อมูลและความสัมพันธ์ในรูปแบบกราฟ โดยจะใช้ node แทนข้อมูลภายใน โดยแต่ละ node จะเก็บ entity ของข้อมูลและใช้ edge แสดงความสัมพันธ์ระหว่างสอง node
- ตัวอย่าง Database Software ที่ใช้งานลักษณะนี้ ได้แก่ ArangoDB, InfiniteGraph, Apache Giraph, MarkLogic, Neo4J, OrientDB, Virtuoso



รูปแบบการจัดเก็บข้อมูล NoSQL (ต่อ)

- แบ่งออกเป็น 4 ประเภทคือ
- 4. Wide-Column Store** เป็นการจัดเก็บข้อมูลในรูปแบบ Tables มี Rows และ Columns แต่จะต่างจาก RDBMS ตรงที่ยืดหยุ่นกว่า โดยแต่ละ Rows จะไม่ Fix Column (ถ้าเป็น RDBMS จะ Fix มาเป็น Pattern เดียวกันทำให้ต้องเก็บข้อมูลในรูปแบบเดียวกันเสมอ)
- ตัวอย่าง Database Software ที่ใช้งานลักษณะนี้ ได้แก่ Amazon DynamoDB, Cassandra, Azure Tables, Accumulo, HBase

AuthorProfile			
Mahesh	Gender	Expertise	Rank
	Male	ADO.NET, C#, GDI+	102
	03182019	03182019	03182019
David	Gender	Book	
	Male	AWS Developer's Guide	
	03202019	03202019	
Allen	City	Book	Rank
	London	Azure Quickstart	89
	04201019	04201019	04201019

<https://www.c-sharpcorner.com/article/what-is-a-column-store-database/>



สรุป

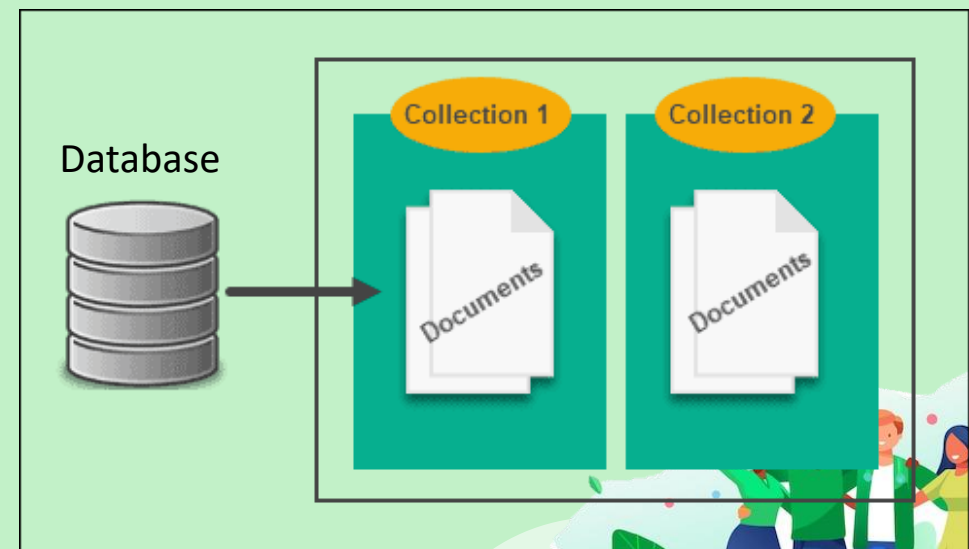
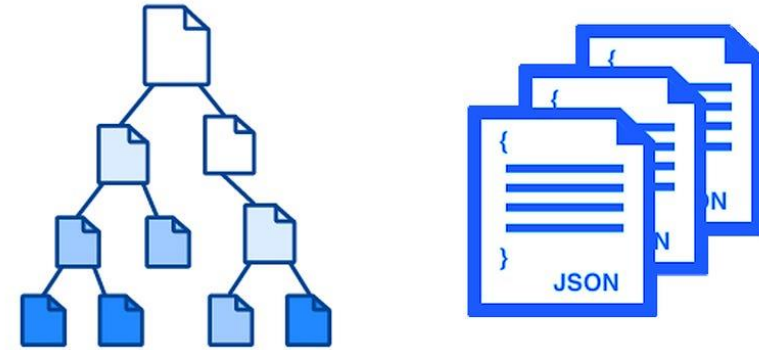
	SQL	NoSQL
รูปแบบ Database	Relational	Non Relational
รูปแบบการจัดเก็บข้อมูล	ตาราง	Key-Value, Document, Graph, Wide-Column
Schema(โครงสร้างฐานข้อมูล)	โครงสร้างแน่นอน	โครงสร้างยืดหยุ่น (ไดนามิก)
การปรับขนาด	แนวตั้ง	แนวนอน
การค้นหา	สำหรับการค้นหาที่ซับซ้อน	สำหรับการค้นหาที่ไม่ซับซ้อน



MongoDB

- เป็น Document Store ซึ่งก็คือการเก็บข้อมูลเชิงเอกสาร
- ใช้สำหรับเก็บข้อมูลขนาดใหญ่ที่มีความยืดหยุ่น งานต่อการปรับขนาดและทำงานข้าม Platform
- การจัดเก็บข้อมูลอยู่ในรูปแบบไฟล์ JSON และบันทึกเก็บเป็นเอกสารไฟล์ BSON
- การจัดเก็บข้อมูลมี 3 ส่วนประกอบ
 - 1. Database เป็นส่วนที่ใช้เก็บชุดข้อมูลหรือ Collection
 - 2. Collection ภายในประกอบด้วยหลาย document
 - 3. Documents เป็นเอกสารที่ใช้เก็บข้อมูลในแต่ละ document ประกอบไปด้วยคู่ของ key-value (คีย์-ค่า)

NoSQL Document Databases



การจัดเก็บข้อมูล MongoDB

Database

Collection

Collection3

Document#1
Key:value

Document#2
{Key:value,
Key:value}

Collection

A Document

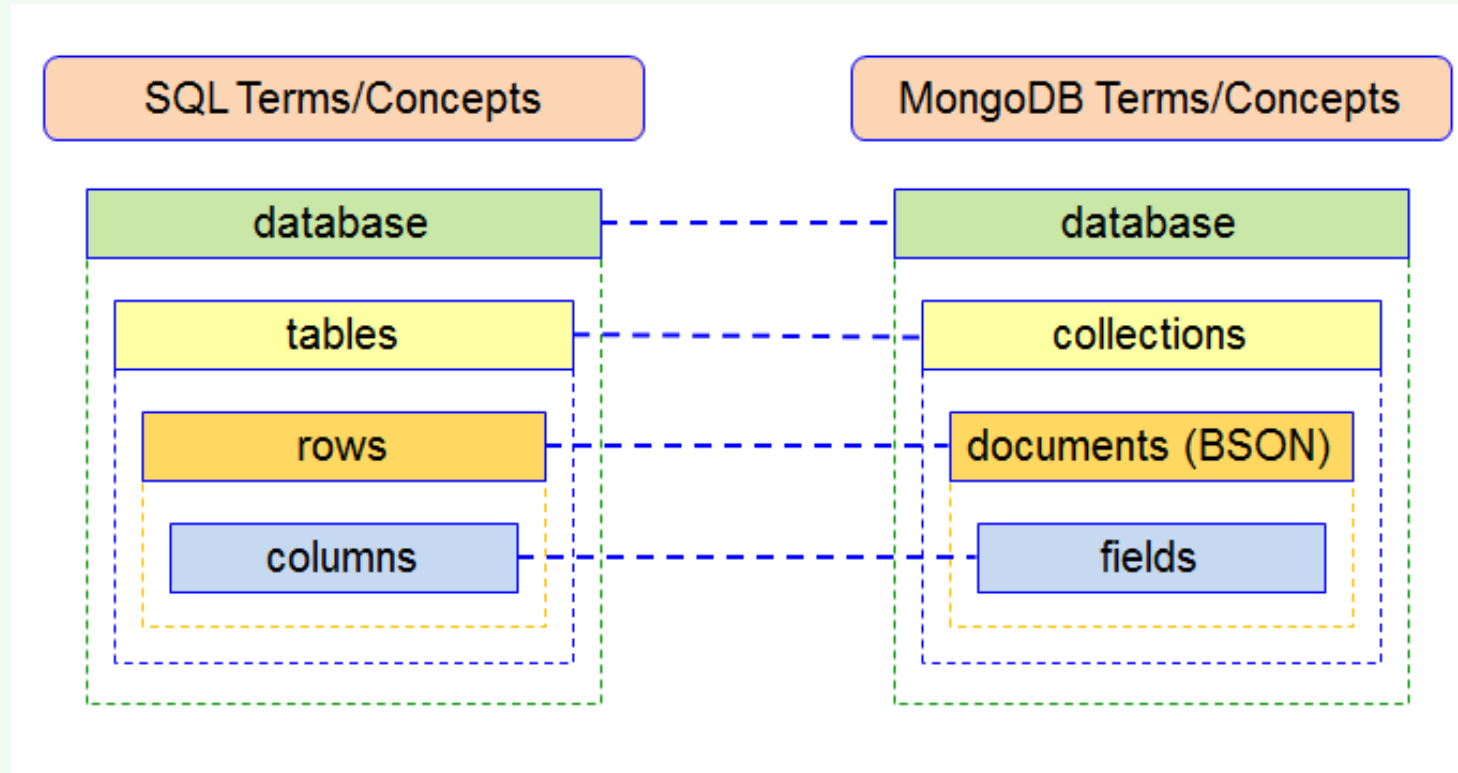
```
{  
  "BookID": "978-1449396091",  
  "Title": "Redis-The Definitive Guide",  
  "Author": "Salvatore Sanfilippo",  
  "Year": "2021",  
}
```

Key	Value
BookID	978-1449396091
Title	Redis - The Definitive Guide
Author	Salvatore Sanfilippo
Year	2021

<https://redis.com/nosql/document-databases/>



RDMS based on SQL and MongoDB



<https://studio3t.com/academy/topic/mongodb-vs-sql-concepts/>



Id	FirstName	LastName	Age	Nationality	AddressLine1	AddressLine2
1	Munir	Niazi	27	Pakistan	Street 1	Street 2
2	William	Wordsworth	21	England	Lane 1	Lane 2
3	Ahmed	Zawail	22	Egypt	Building A	Building B



```
{
  "_id": ObjectId("5c237223efce7b2e7b4e49ff"),
  "firstName": "Munir",
  "lastName": "Niazi",
  "age": 27,
  "nationality": "Pakistan",
  "address": {
    line1: "Street 1",
    line2: "Street 2"
  }
},
{
  "_id": ObjectId("5c237223efce7b2e7b4e4a00"),
  "firstName": "William",
  "lastName": "Wordsworth",
  "age": 21,
  "nationality": "England",
  "address": {
    line1: "Lane 1",
    line2: "Lane 2"
  }
},
{
  "_id": ObjectId("5c237223efce7b2e7b4e4a01"),
  "firstName": "Ahmed",
  "lastName": "Zawail",
  "age": 22,
  "nationality": "Egypt",
  "address": {
    line1: "Building A",
    line2: "Building B"
  }
}
```

JSON

- JSON ย่อมาจาก (JavaScript Object Notation) เป็นรูปแบบที่ใช้ในการแลกเปลี่ยนข้อมูล (Data Interchange Format) ระหว่างเซิร์ฟเวอร์และไคลเอนต์ที่ได้รับความนิยมในปัจจุบัน ในอดีตการแลกเปลี่ยนหรือรับส่งข้อมูลนั้นจะใช้รูปแบบ XML ที่มีโครงสร้างซับซ้อนและขนาดใหญ่ จึงมีการเปลี่ยนมาใช้รูปแบบ JSON เนื่องจาก อ่านทำความเข้าใจได้ง่าย มีความเบา (lightweight) มีความเร็วในการ access ข้อมูลที่สูง เพราะไม่ได้มีโครงสร้างที่ซับซ้อนเหมือน XML
- อักขระมาตรฐานของ JSON
 - เครื่องหมาย : ใช้สำหรับแยกค่า key และ value
 - เครื่องหมาย , ใช้สำหรับแยกข้อมูล key-value ในแต่ละคู่
 - เครื่องหมาย { และ } ระบุว่าข้อมูลเป็นออบเจ็กต์
 - เครื่องหมาย [และ] ระบุว่าข้อมูลเป็นอาร์เรย์
 - เครื่องหมาย “ ” (double quotes) ใช้เขียนค่า key-value ใน JSON



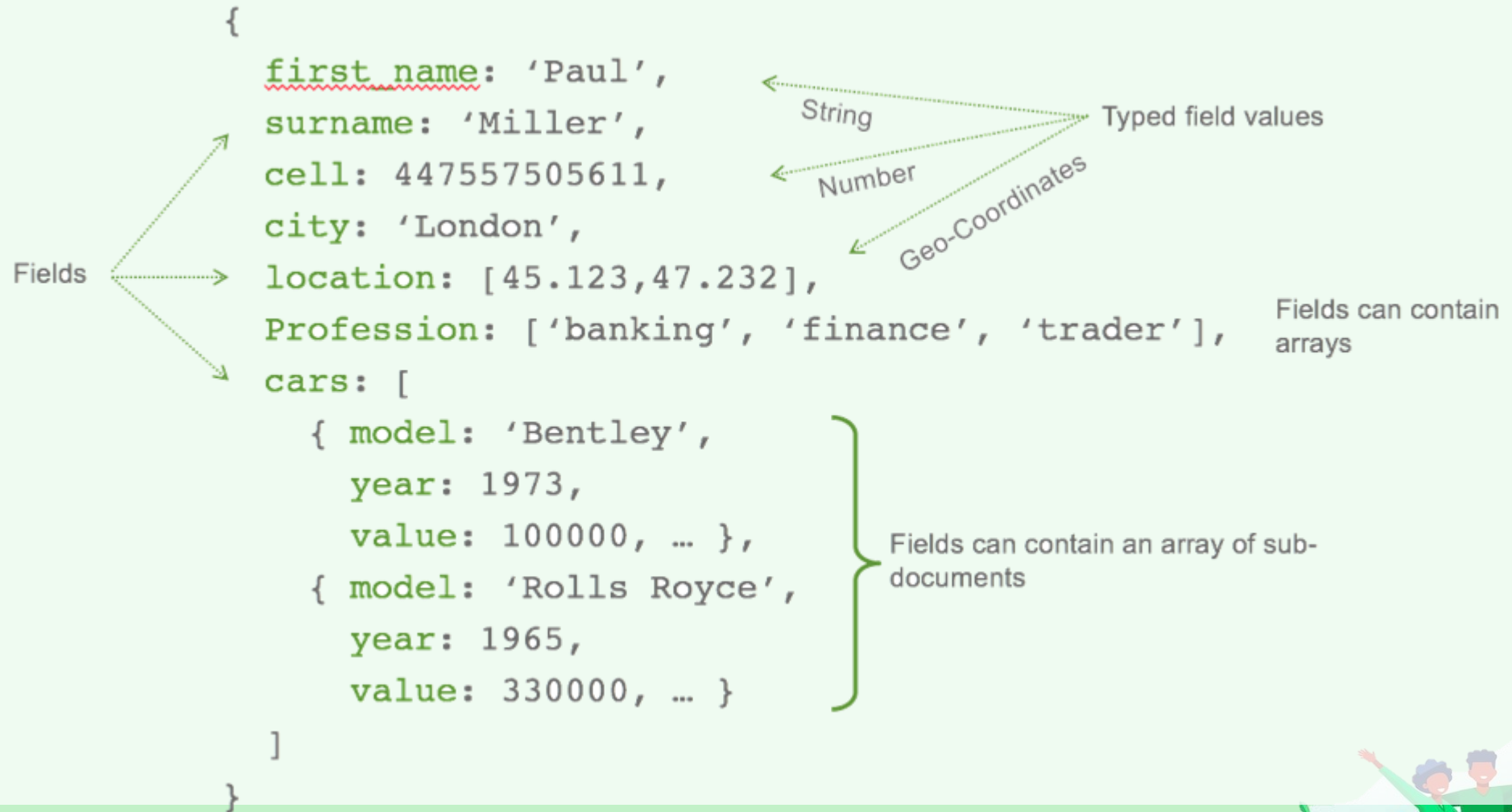
MongoDB Documents: Fields & Data Types

- Document จะเริ่มต้นด้วยปีกกาเปิด-ปิด { }
- ข้อมูลแต่ละ field จะคั่นด้วยลูกน้ำ ใน 1 field สามารถมีค่าได้ 1 ค่า หรือมากกว่าก็ได้
- สามารถสร้าง Document ย่อยภายใน Document ได้

```
db.users.insert (  ← collection
{
  name: "sue",      ← field: value
  age: 26,          ← field: value
  status: "A"       ← field: value
}                  } document
)
```



MongoDB Documents: Fields & Data Types



JSON to BSON

- ในฐานข้อมูล MongoDB จะแปลง JSON ให้เข้ารหัสอยู่ในรูปแบบไบนารีหรือเป็น BSON (Binary JSON) เพื่อขยายความสามารถให้ JSON มีชนิดข้อมูลเพิ่มมากขึ้น มีการเข้าและถอดรหัสด้วยภาษาที่แตกต่างกันได้
- BSON ทำงานรวดเร็วและมีขนาดเบาเหมือน JSON
- MongoDB เข้าถึงข้อมูลหรือ Object ใน BSON เพื่อสร้าง index ในการทำระบบสอบถามข้อมูล (Query) โดย BSON จะทำงานอยู่เบื้องหลัง ส่วนการแสดงผลข้อมูลกับผู้ใช้จะเป็น JSON

JSON

```
{  
  "array": [  
    1,  
    2,  
    3  
  ],  
  "boolean": true,  
  "null": null,  
  "number": 123,  
  "object": {  
    "a": "b",  
    "c": "d",  
    "e": "f"  
  },  
  "string": "Hello World"  
}
```

BSON

{ 01010100
11101011
10101110
01010101 }



การติดตั้ง MongoDB

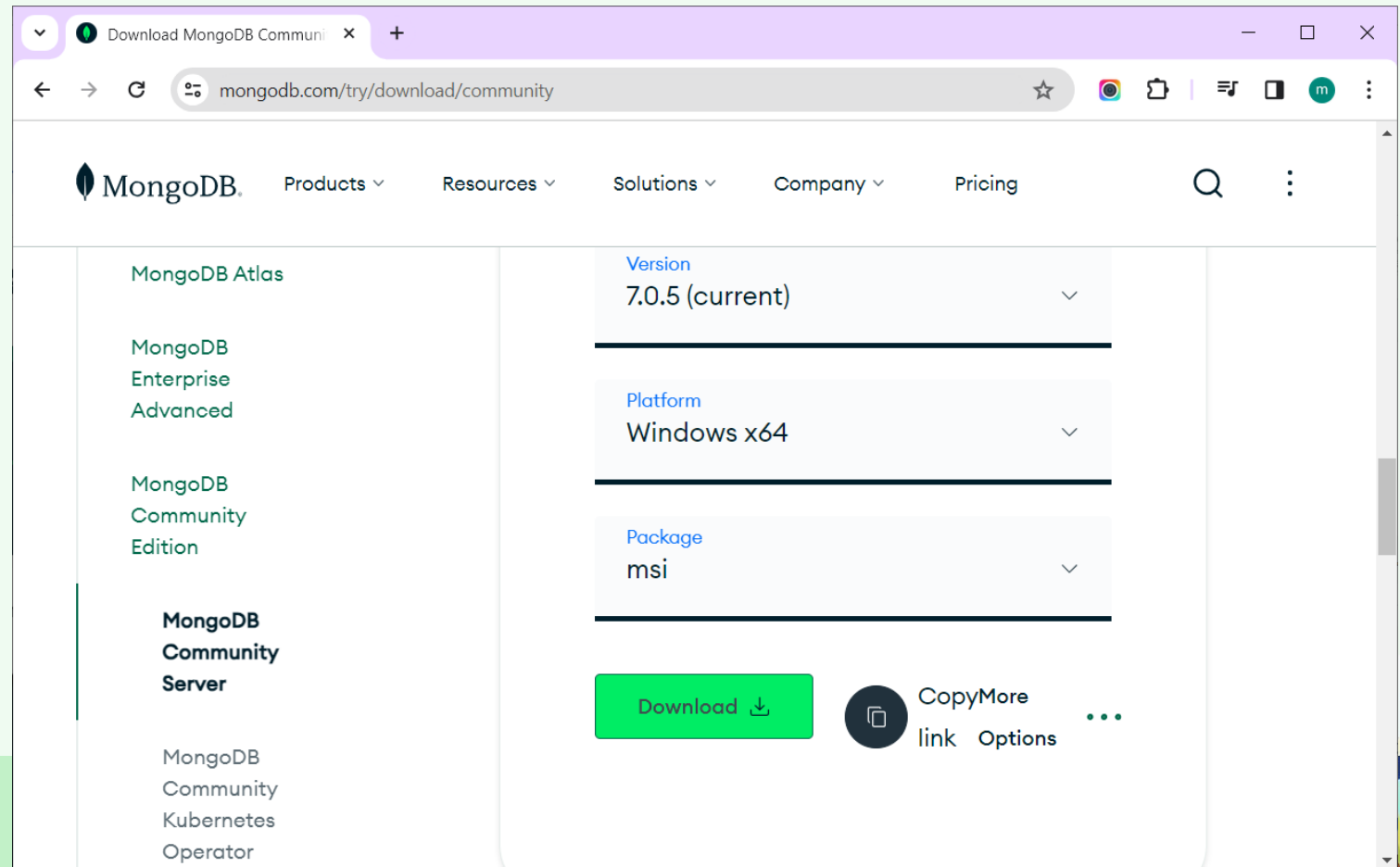
- MongoDB Community เป็น MongoDB Server รุ่น Open Source ที่ให้ดาวน์โหลดใช้งานฟรี
- MongoDB Compass เป็นเครื่องมือในการจัดการข้อมูลในรูปแบบ GUI เพื่ออำนวยความสะดวกในการวิเคราะห์ข้อมูล การดำเนินการกับข้อมูล (CRUD) รวมทั้งระบบสืบค้นและจัดการ index



การติดตั้ง MongoDB Community และ MongoDB Compass

22

- เข้า <https://www.mongodb.com/try/download/community>
- ให้เลือก version, platform, package แล้วกด Download
- เปิดไฟล์ที่ดาวน์โหลด



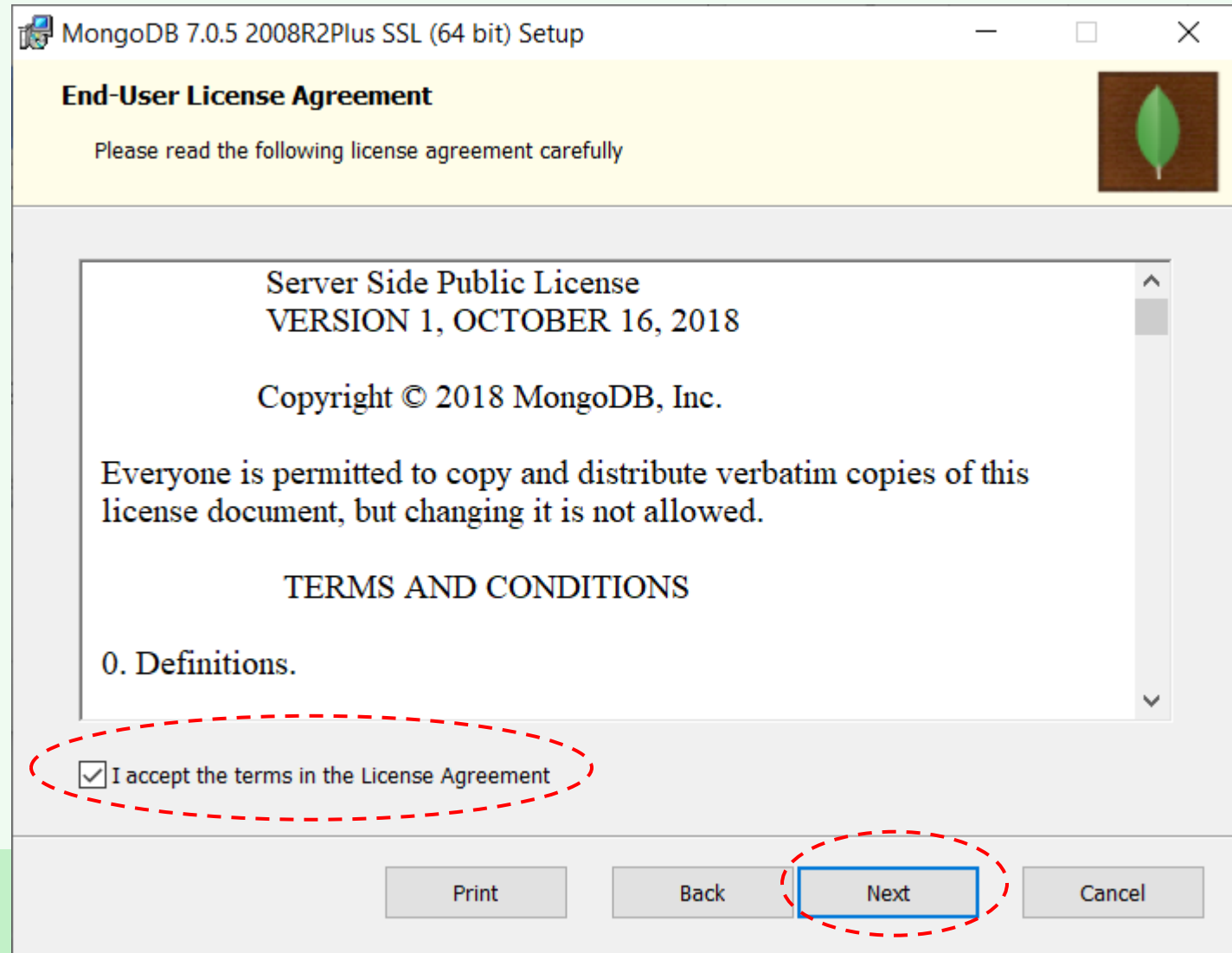
การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- กด Next



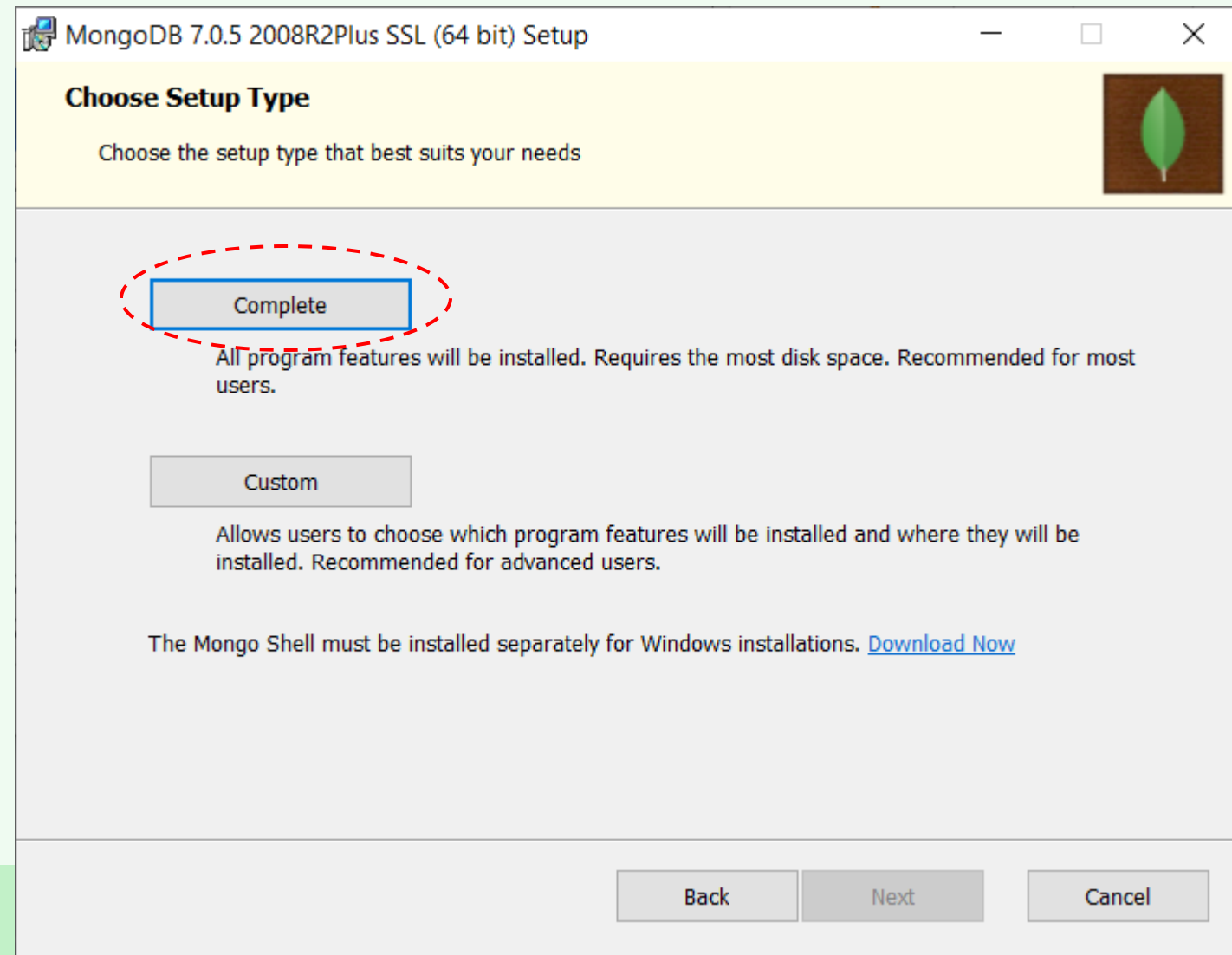
การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- เลือก I accept... แล้ว กด Next



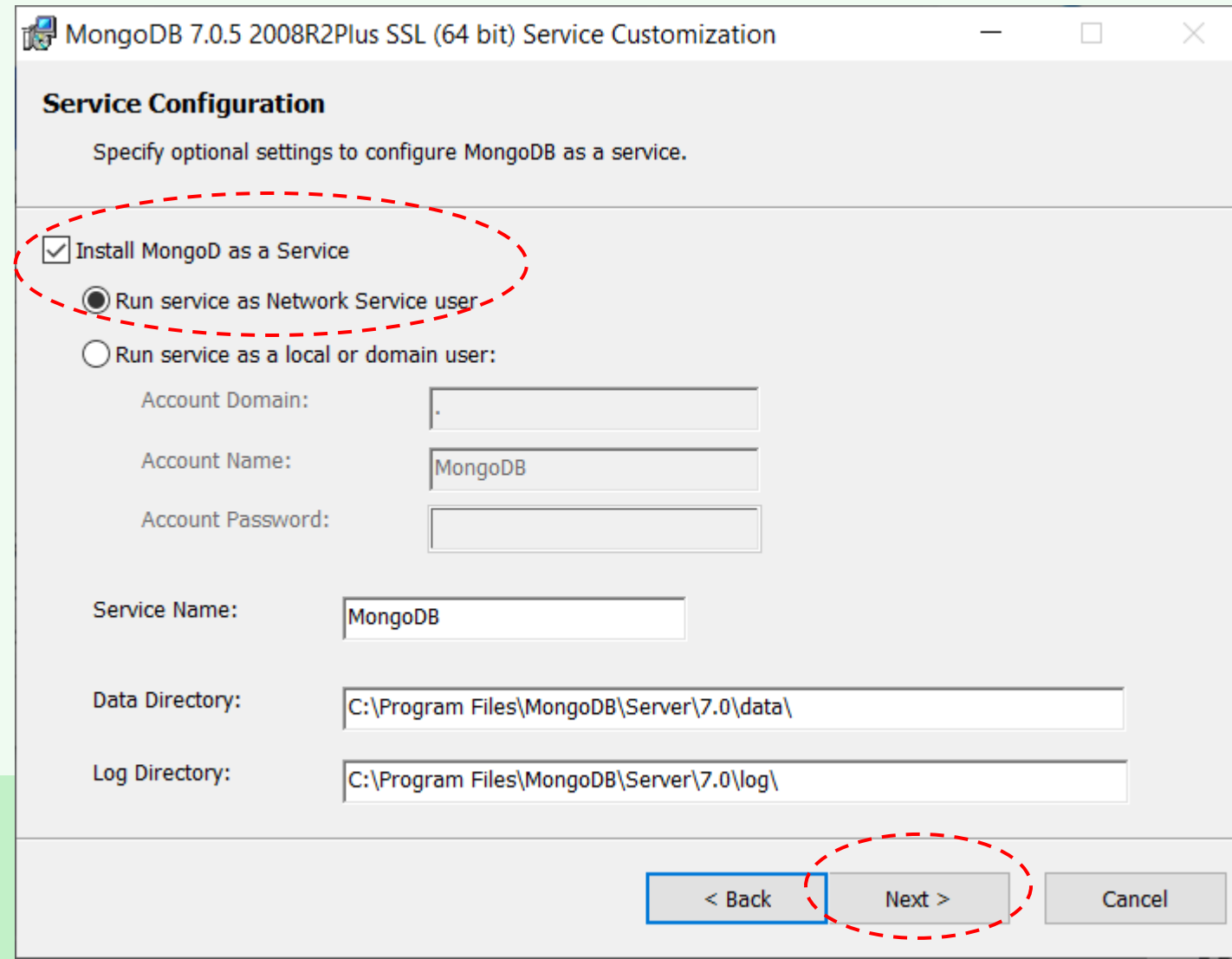
การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- เลือก Complete



การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- เลือก Run service as Network Service user เพราะไม่ต้องการสร้าง user และ password ในการทำงานของ MongoDB
- กด Next



MongoDB 7.0.5 2008R2Plus SSL (64 bit) Service Customization

Service Configuration
Specify optional settings to configure MongoDB as a service.

☒ Install MongoDB as a Service

☒ Run service as Network Service user

☐ Run service as a local or domain user:

Account Domain: .

Account Name: MongoDB

Account Password:

Service Name: MongoDB

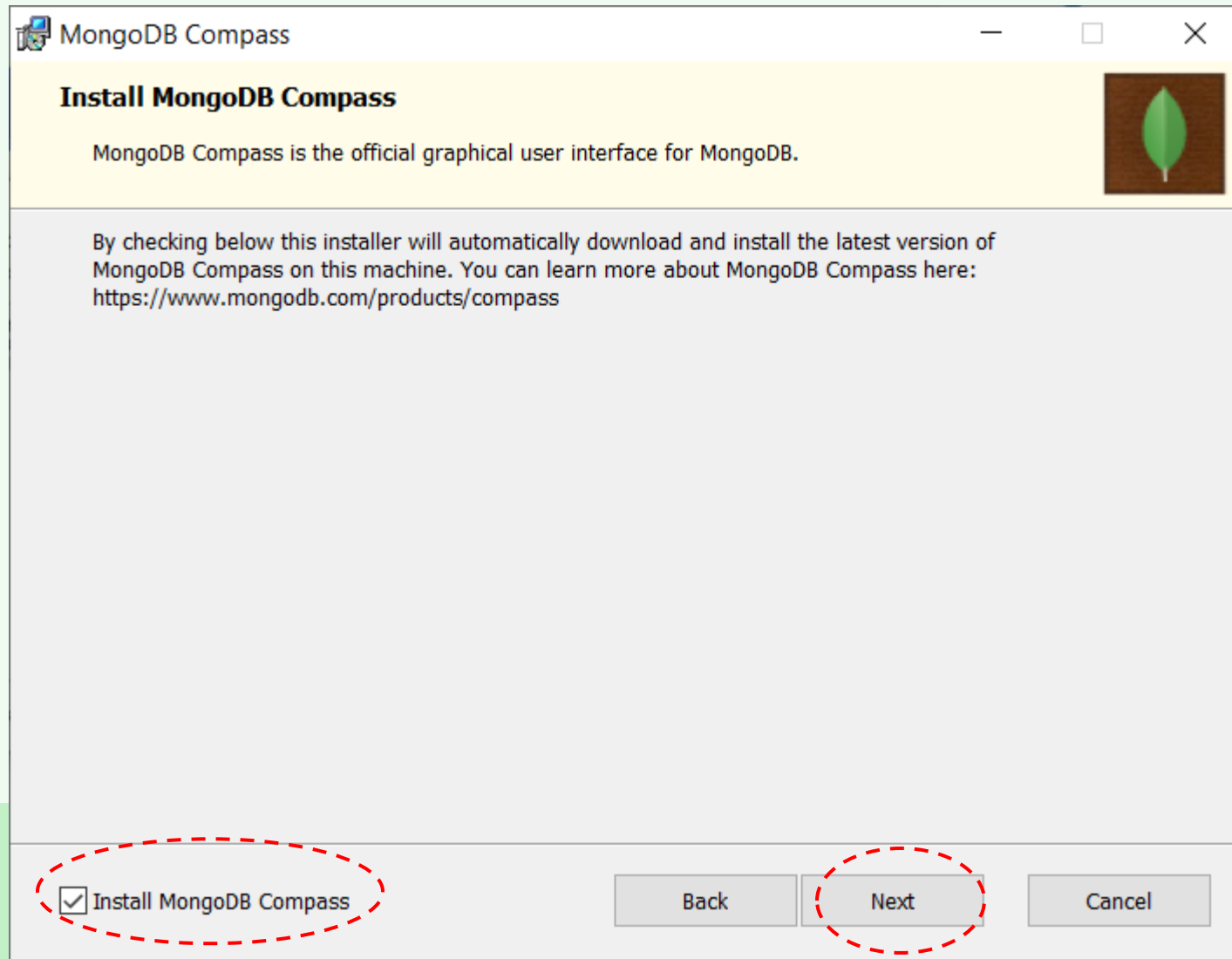
Data Directory: C:\Program Files\MongoDB\Server\7.0\data\

Log Directory: C:\Program Files\MongoDB\Server\7.0\log\

< Back Next > Cancel

การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- เลือก Install MongoDB Compass
- กด Next



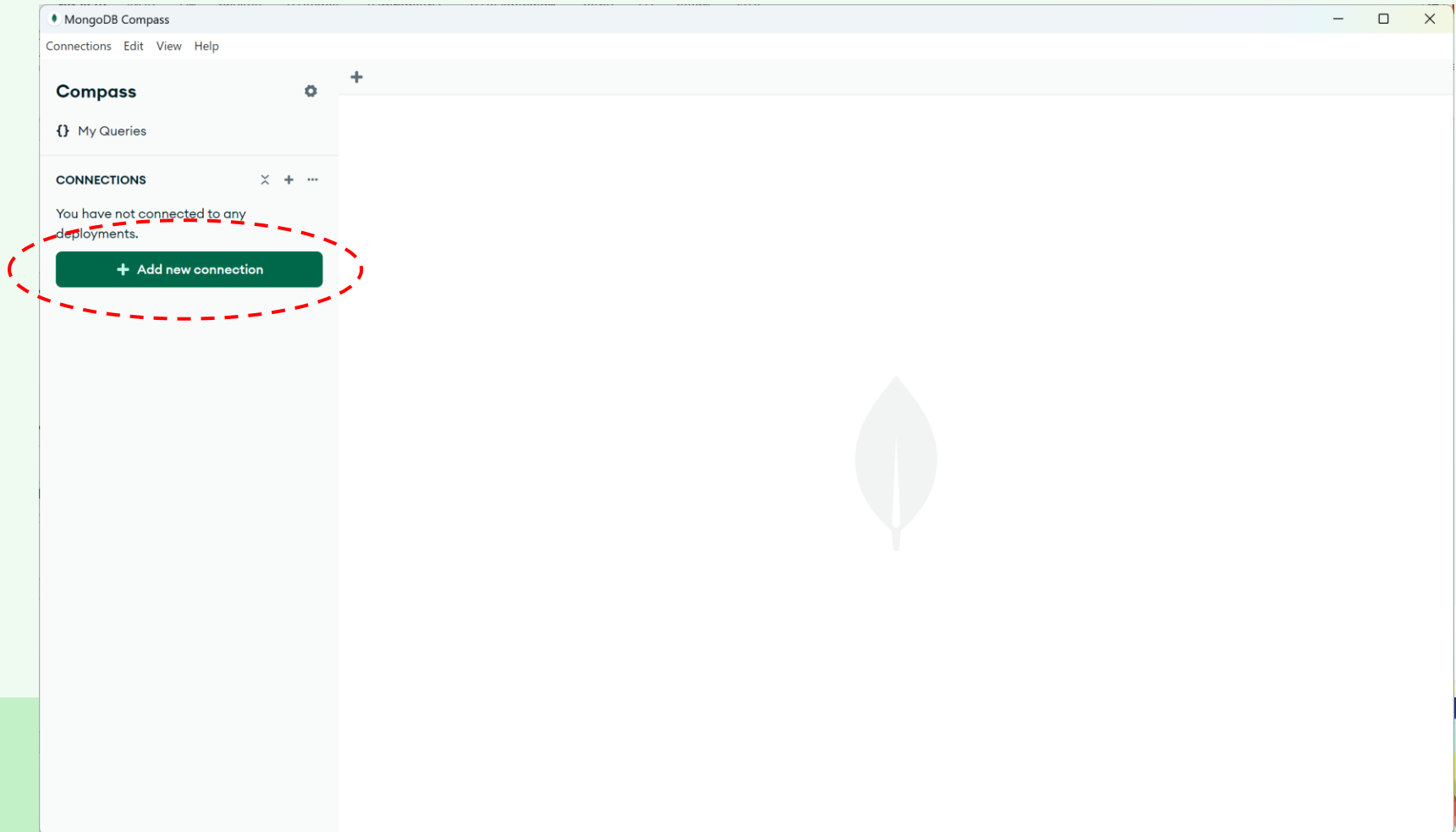
การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- เมื่อโปรแกรมติดตั้งเสร็จเรียบร้อยแล้ว
- กด Finish



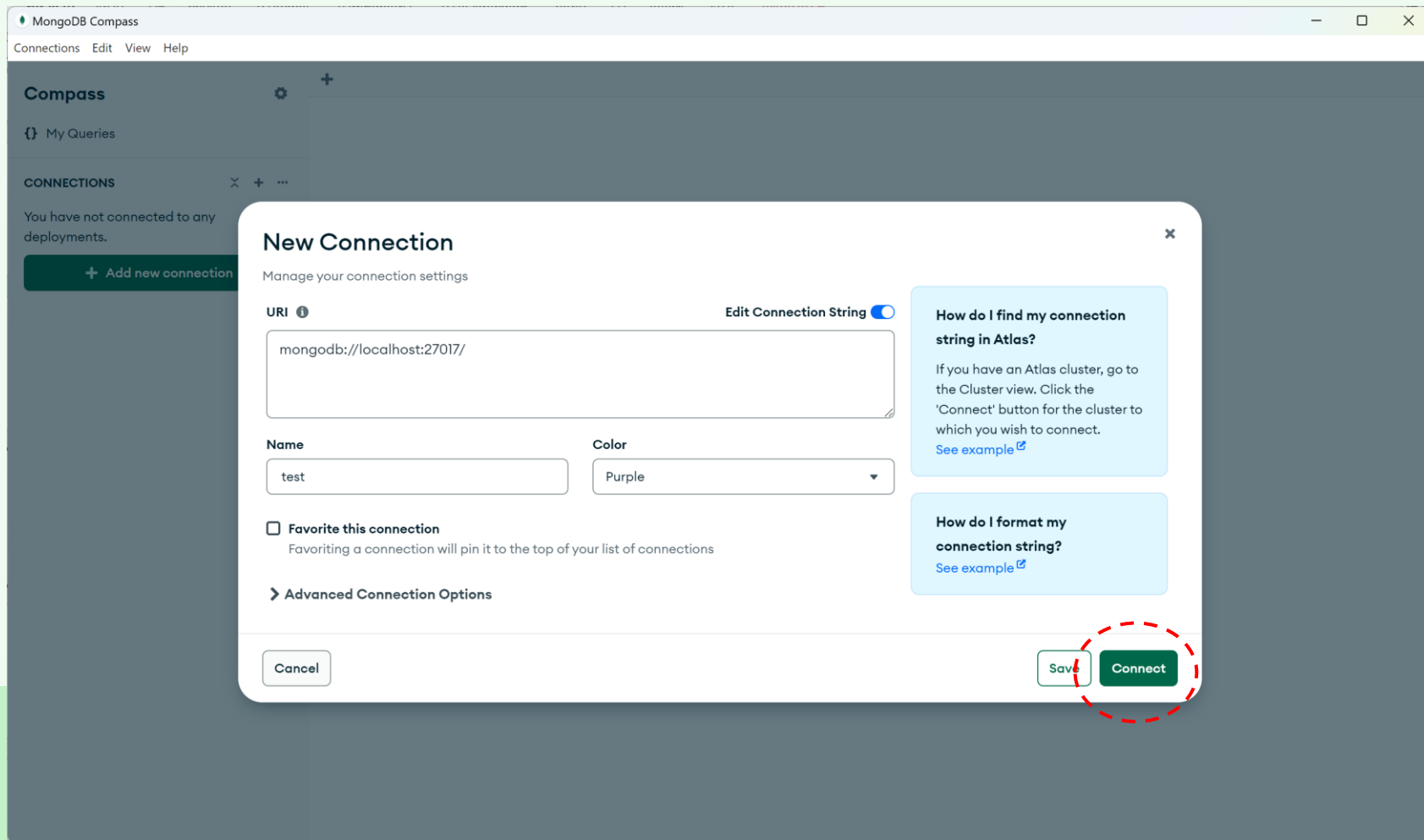
การติดตั้ง MongoDB Community และ MongoDB Compass (ต่อ)

- เมื่อลงโปรแกรมติดตั้งเสร็จเรียบร้อยแล้ว จะแสดงหน้าต่างโปรแกรม MongoDB Compass
- ให้สร้าง Connection โดยกด Add new connection



เริ่มต้นใช้งาน MongoDB

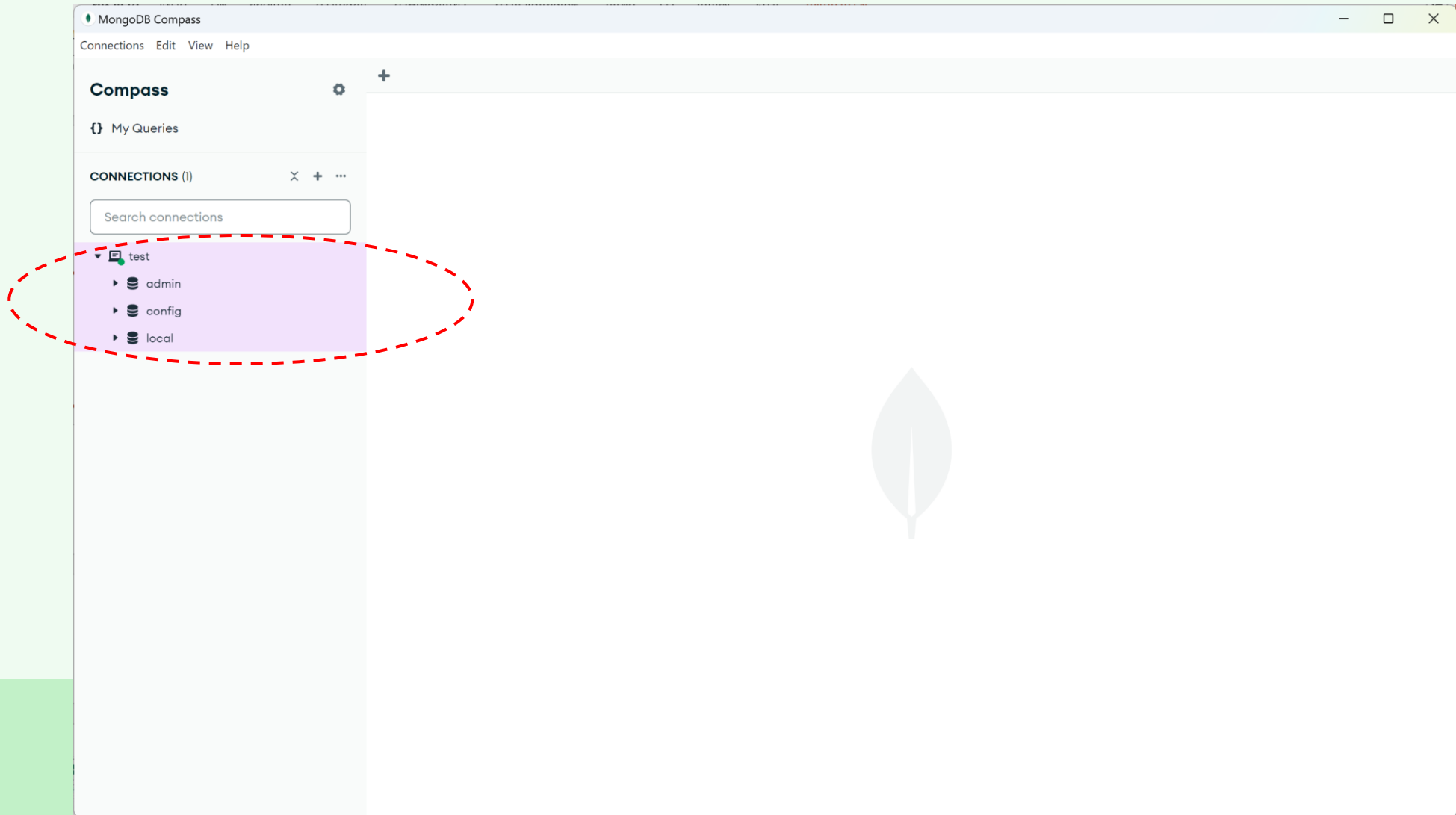
- สามารถกำหนด port ในการเชื่อมต่อ MongoDB ตั้งชื่อและกำหนดสีที่แถบเมนู
- กด Connect เมื่อต้องการใช้งาน MongoDB



เริ่มต้นใช้งาน MongoDB (ต่อ)

31

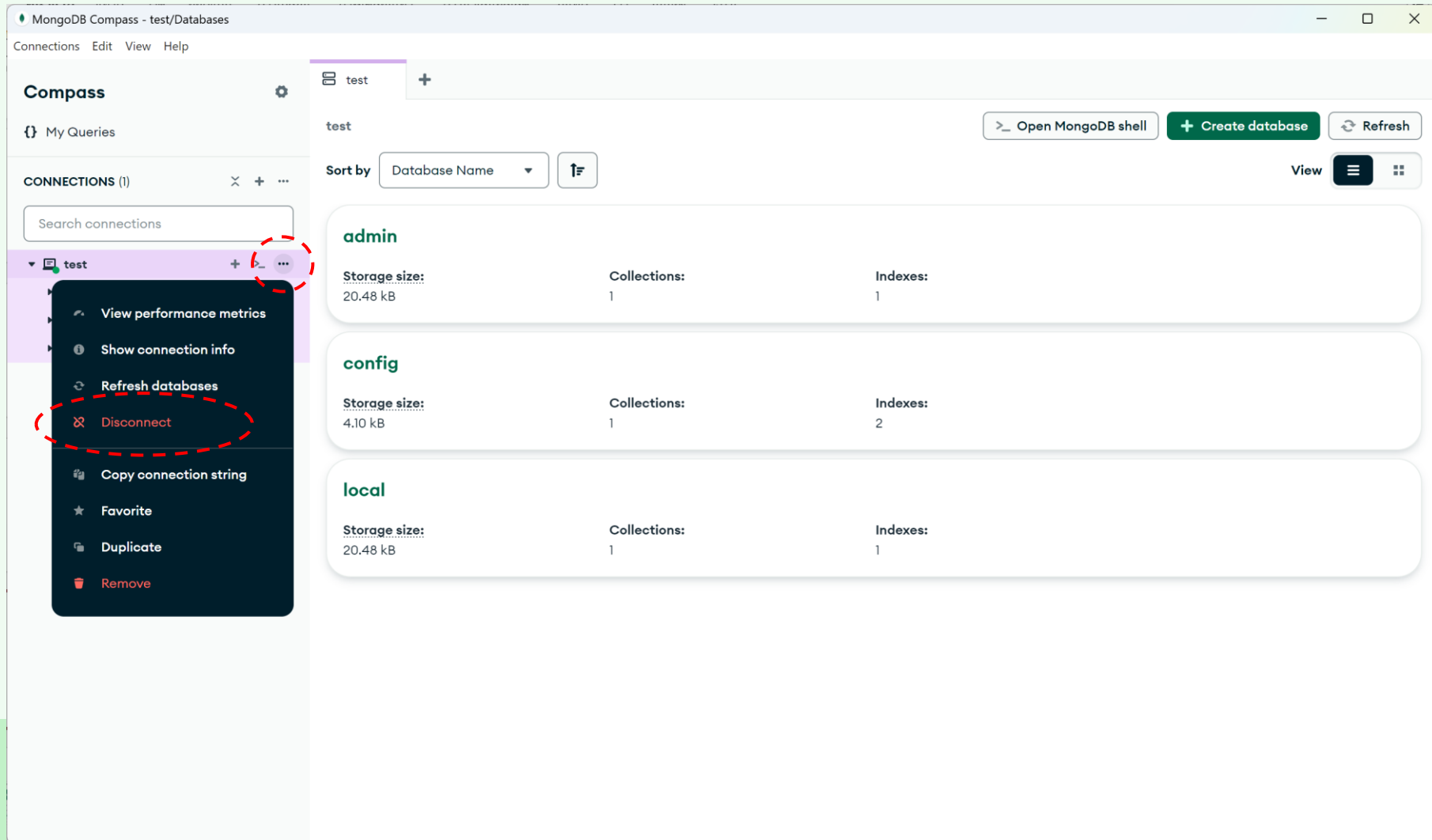
- จะแสดง DB ที่สร้าง



เมื่อเลิกใช้งาน MongoDB

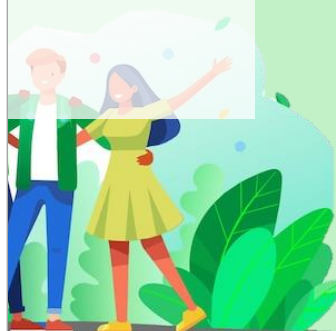
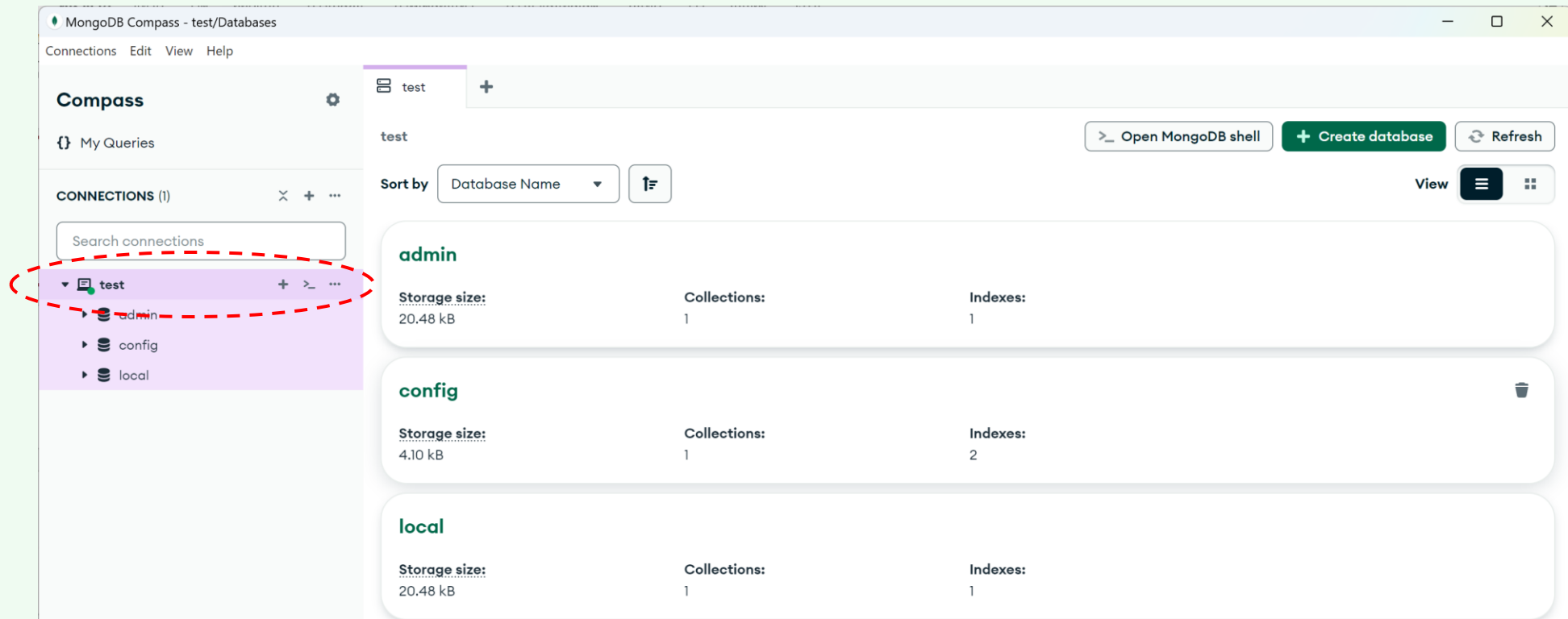
32

- ไปที่แถบเมนู เลือก Connect -> Disconnect



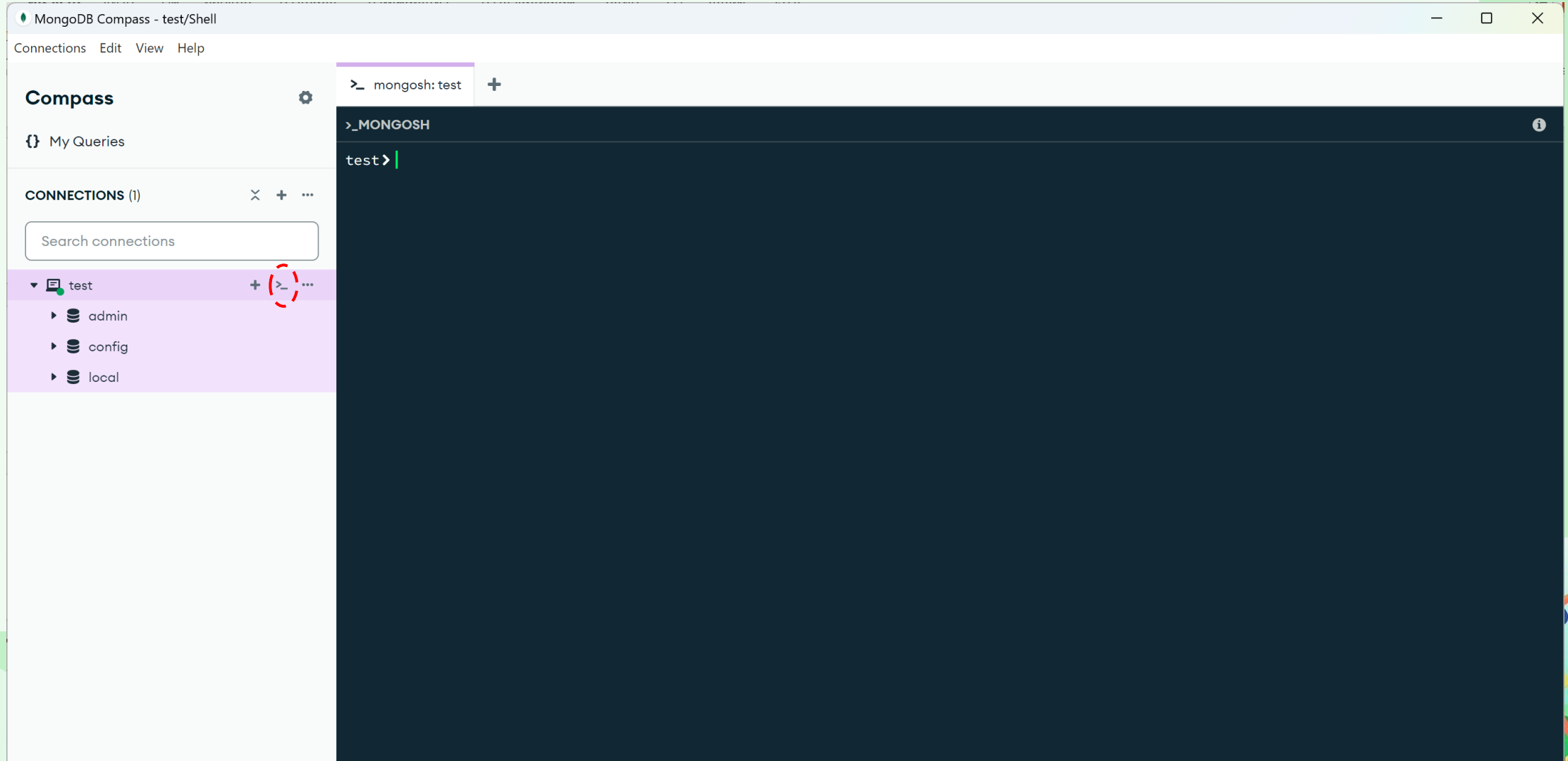
ฐานข้อมูลใน MongoDB

- เลือกเมนู Databases จะแสดงรายชื่อฐานข้อมูลที่มีในระบบ



การดำเนินการในฐานข้อมูลด้วย Mongo Shell

- ให้กด >_ เพื่อแสดง Mongo Shell



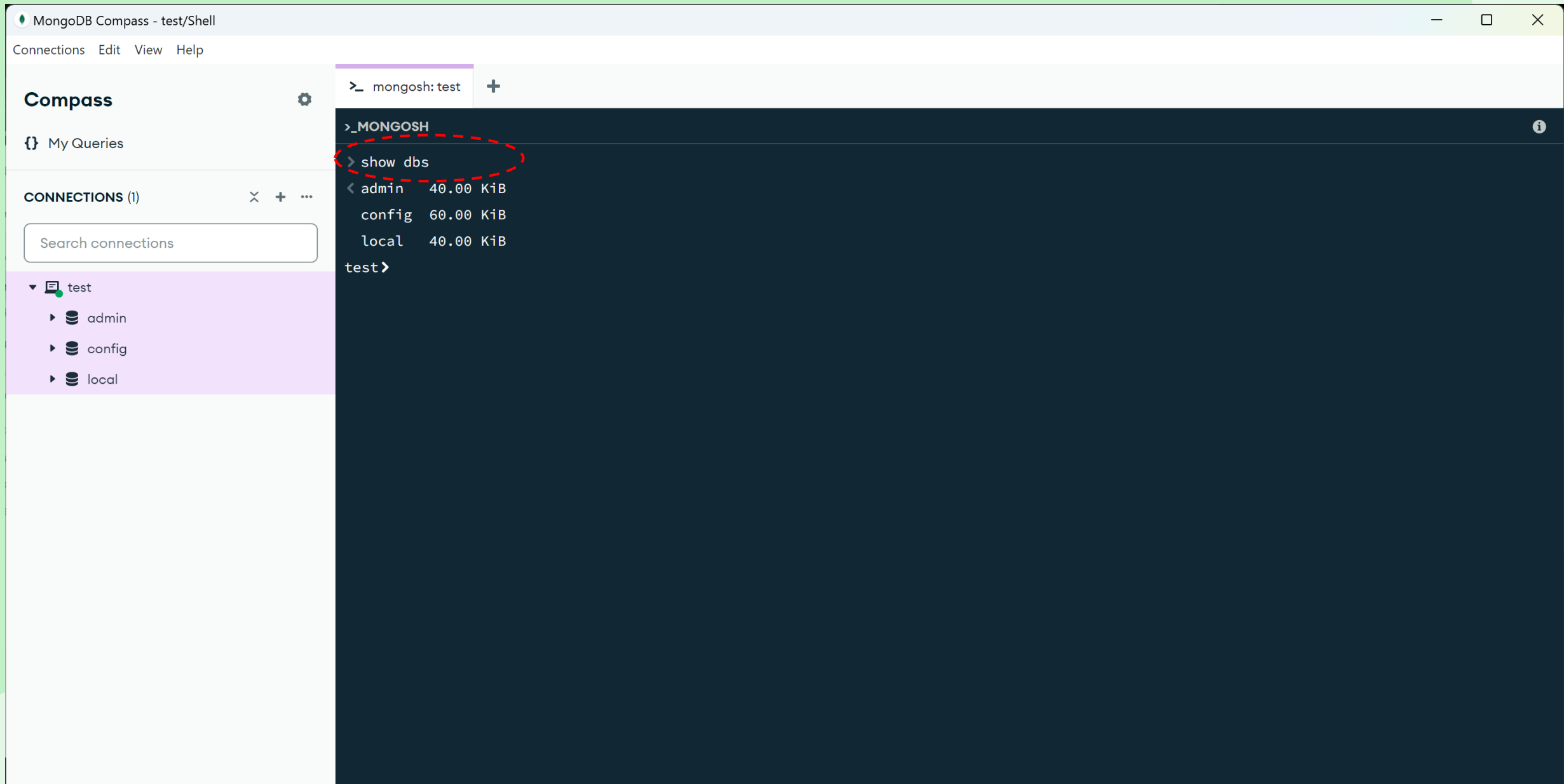
คำสั่งที่ใช้ในการสร้างและลบฐานข้อมูล

- แสดงฐานข้อมูล
 - show dbs
- สร้างหรือเลือกฐานข้อมูล
 - use ชื่อฐานข้อมูล
 - โดย
 - ถ้า ชื่อฐานข้อมูล ยังไม่มีในระบบจะเป็นการสร้างฐานข้อมูลขึ้นมา
 - ถ้า ชื่อฐานข้อมูล มีในระบบจะเป็นการเลือกฐานข้อมูลมาใช้
- ลบฐานข้อมูล
 - db.dropDatabase()
 - โดยจะต้องเข้าไปในฐานข้อมูลก่อน ถึงจะลบได้



ตัวอย่างการใช้คำสั่ง show dbs เพื่อแสดงฐานข้อมูลทั้งหมด

36

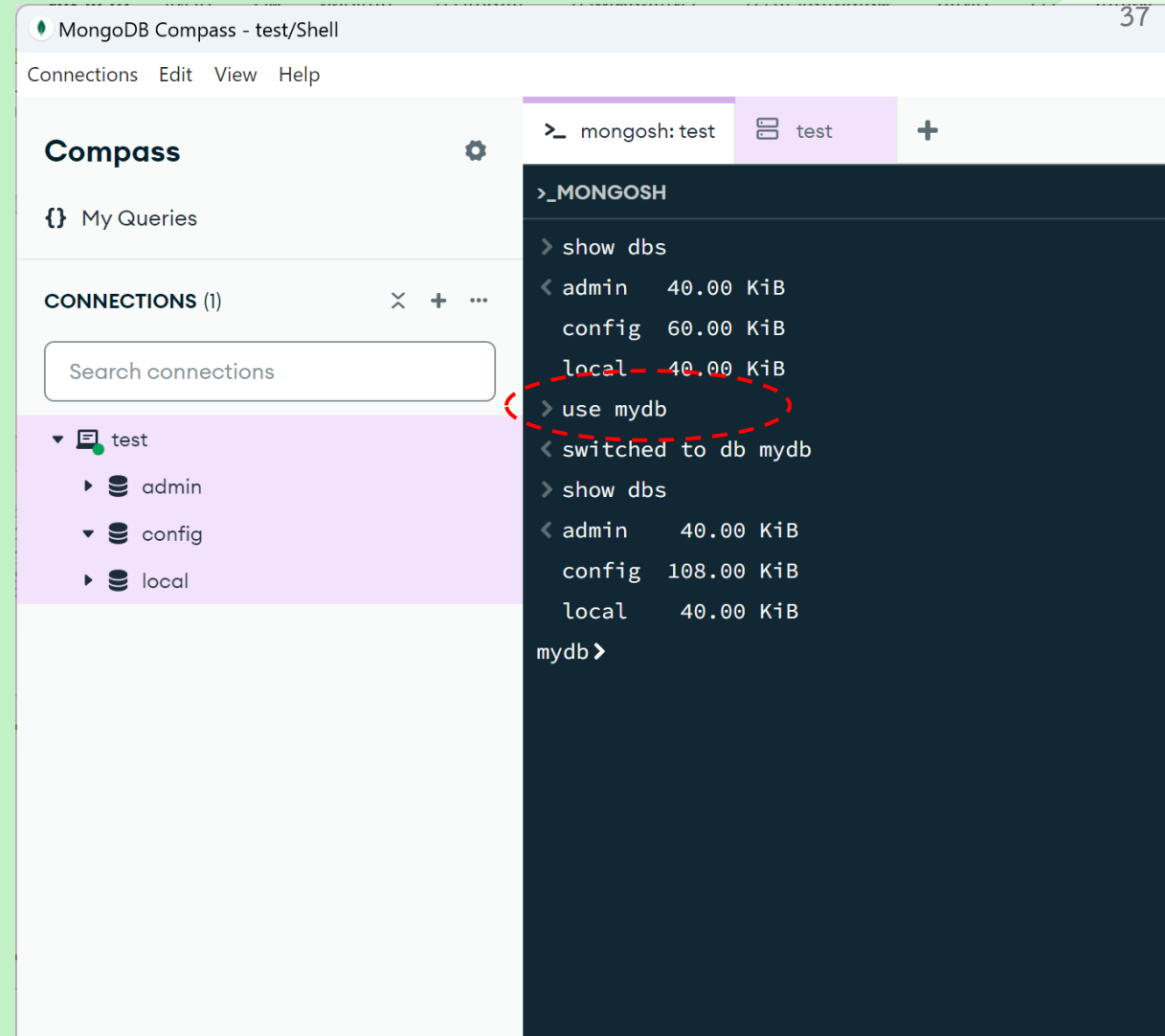


The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists the 'test' database with its sub-databases: 'admin', 'config', and 'local'. The main area displays the 'mongosh: test' shell. The command '> show dbs' is entered and highlighted with a red dashed circle. The output shows the size of each database: 'admin' (40.00 KiB), 'config' (60.00 KiB), and 'local' (40.00 KiB). The prompt 'test>' is visible at the bottom of the shell.

```
>_MONGOSH
> show dbs
< admin  40.00 KiB
  config 60.00 KiB
  local  40.00 KiB
test>
```

ตัวอย่างการใช้คำสั่งสร้างหรือเลือกฐานข้อมูล

- ใช้ use mydb เพื่อสร้างฐานข้อมูล mydb เนื่องจากในระบบไม่มีฐานข้อมูลชื่อนี้
- เมื่อกด Enter จะแสดงข้อความ switched to db mydb เพื่อบอกว่าตอนนี้ได้เข้าใช้งาน mydb แล้ว
- mydb> ที่แสดงในรูป เพื่อบอกว่าตอนนี้ทำงานอยู่ที่ mydb
- เลือกเมนู Databases จะยังไม่มีชื่อฐานข้อมูล mydb ในระบบเพราะ mydb ยังไม่มี collection
- ถ้าต้องการ clear หน้าต่าง Mongo Shell ให้พิมพ์ cls



คำสั่งที่ใช้ในการจัดการ collection

- สร้าง Collection
 - use ชื่อฐานข้อมูล
 - `db.createCollection("ชื่อCollection")`
 - โดยจะต้องเข้าไปในฐานข้อมูลก่อน ถึงจะสร้าง Collection ได้
- เปลี่ยนชื่อ Collection (ต้องเลือก ฐานข้อมูล ก่อน)
 - `db.ชื่อCollection.renameCollection("ชื่อCollectionใหม่")`
- ลบ Collection (ต้องเลือก ฐานข้อมูล ก่อน)
 - `db.ชื่อCollection.drop()`
- แสดง Collection ทั้งหมด (ต้องเลือก ฐานข้อมูล ก่อน)
 - `show collections`



ชนิดข้อมูลใน MongoDB

ชนิด	รายละเอียด
String	จัดเก็บข้อความ ข้อความจะต้องอยู่ภายใน “ ”
Integer	เลขจำนวนเต็ม
Double	เลขทศนิยม
Boolean	True/False
Array	เก็บชุดข้อมูลโดยเขียนในวงเล็บ []
Object	เก็บ document โดยเขียนในปีกกา { }
Date	วันที่ สามารถใช้ฟังก์ชัน เช่น Date() คื้นค่าเป็น string, ISODate() คื้นค่าเป็น object
Timestamp	วันที่และเวลา ใช้ Timestamp()
Null	ค่า null หรือไม่มีค่า
Binary	ข้อมูลแบบ Binary
Max/Min key	เปรียบเทียบค่ามากกว่า / น้อยกว่า กับข้อมูลอื่น

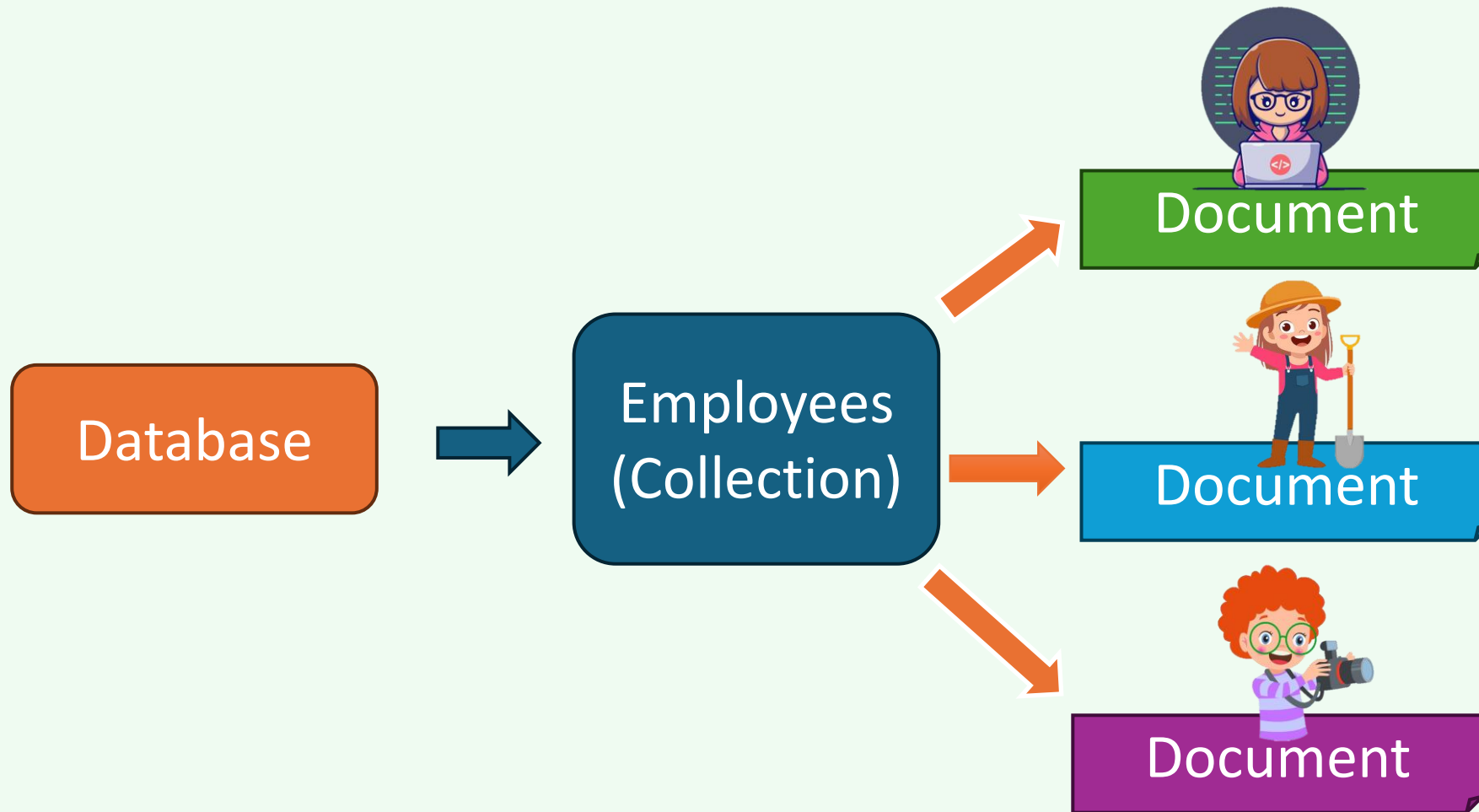
คำสั่งที่ใช้ในการเพิ่ม Document

- เพิ่ม Document
 - `db.ชื่อCollection.insertOne(<Document>)`
- เพิ่มหลาย Document
 - `db.ชื่อCollection.insertMany([<Document>])`



ตัวอย่าง การสร้าง Employees (Collection) พร้อมใส่ Document

41



ก่อนเพิ่ม Document จะต้องเตรียมรูปแบบ Document ก่อน

```
{  
  "employee_name": "Edward Bill",  
  "employee_skills": ["Software Development", "C++", "Java"]  
}
```

รูปแบบ Document

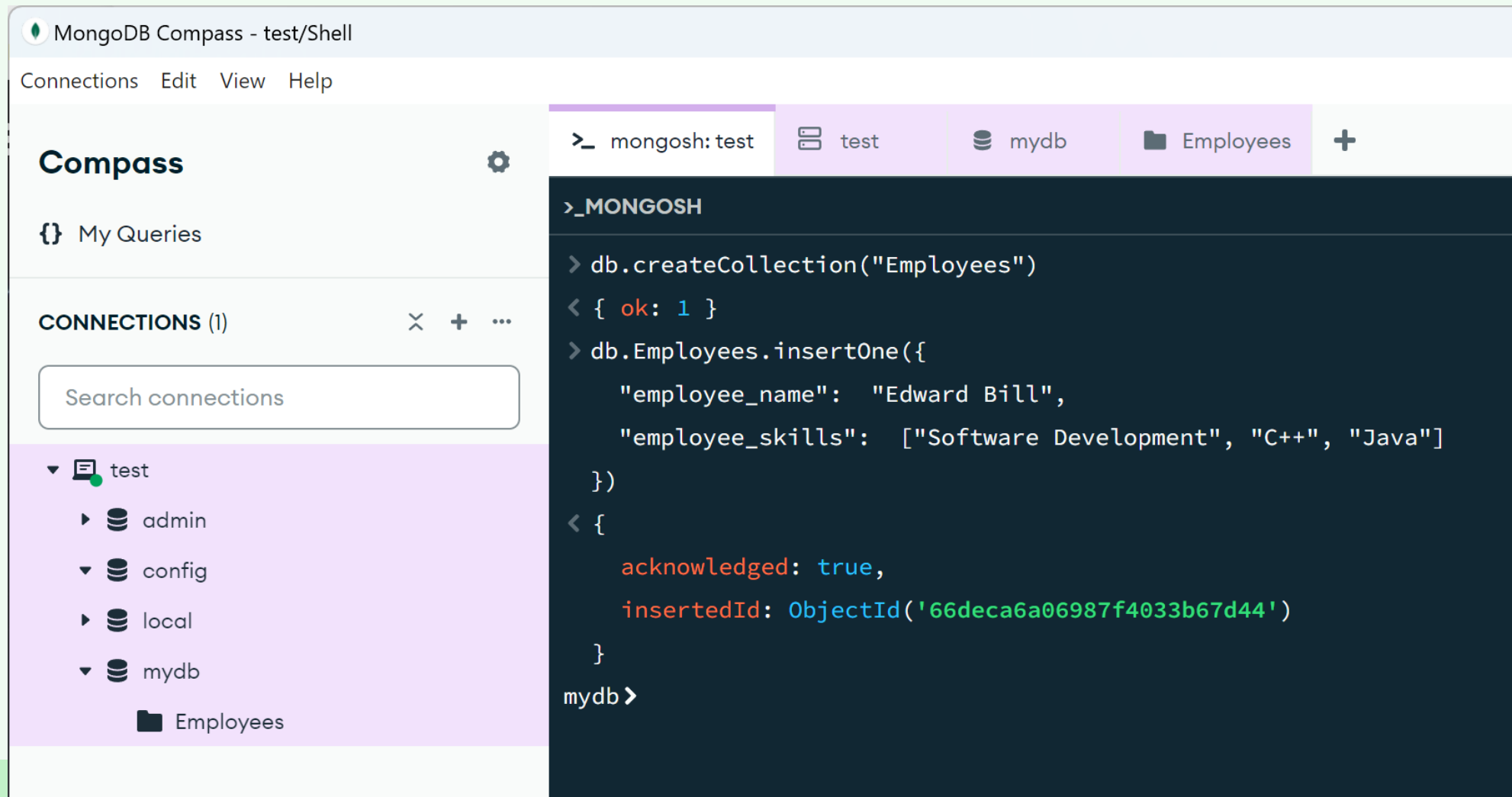


เพิ่ม Document ด้วยคำสั่ง

```
db.Employees.insertOne({  
  "employee_name": "Edward Bill",  
  "employee_skills": ["Software Development", "C++", "Java"]  
})
```

ตัวอย่าง การสร้าง Employees (Collection) พร้อมใส่ Document

43



The screenshot displays the MongoDB Compass application interface. On the left sidebar, the 'test' database is selected, and the 'Employees' collection is visible under the 'mydb' database. The main panel shows the MongoDB Shell with the following commands and output:

```
>_ mongosh: test
>_MONGOSH
> db.createCollection("Employees")
< { ok: 1 }
> db.Employees.insertOne({
  "employee_name": "Edward Bill",
  "employee_skills": ["Software Development", "C++", "Java"]
})
< {
  acknowledged: true,
  insertedId: ObjectId('66deca6a06987f4033b67d44')
}
mydb>
```



ตัวอย่าง การสร้าง Employees (Collection) พร้อมใส่ Document หลายอัน

- ใช้ [] มาครอบ Document เช่น

```
[{  
  "employee_name": "Edward Bill",  
  "employee_skills": ["Software Development", "C++", "Java"]  
},  
{  
  "employee_name": "Jojo Lee",  
  "employee_skills": ["Java", "MySQL"]  
}]
```

รูปแบบ Document



เพิ่ม Document ด้วยคำสั่ง

```
db.Employees.insertMany(  
  [{"employee_name": "Edward Bill", "employee_skills": ["Software Development", "C++", "Java"]},  
  {"employee_name": "Jojo Lee", "employee_skills": ["Java", "MySQL"]}]  
)
```

บันทึกข้อมูลแบบ array

- การกำหนด field ให้มีรูปแบบ array เพื่อเก็บข้อมูลได้หลายค่า โดยจะกำหนดข้อมูลใน []

```
{  
  "employee_name": "Edward Bill",  
  "employee_skills": ["Software Development", "C++", "Java"]  
}
```

string

array

รูปแบบ Document



เพิ่ม Document ด้วยคำสั่ง

```
db.Employees.insertOne({  
  "employee_name": "Edward Bill",  
  "employee_skills": ["Software Development", "C++", "Java"]  
})
```

บันทึกข้อมูลแบบ object

- ใช้งานง่ายกว่า array เพราะไม่ต้องจำว่า index ที่เท่าไรคืออะไร ข้อมูลจะอยู่ใน { } และ ภายในประกอบด้วยคู่ของข้อมูล ชื่อฟิลด์ย่อย:ค่า

```
{  
  "employee_name": "Edward Bill",  
  "employee_skills": {skill1:"Software Development", skill2:"C++", skill3:"Java"}  
}
```

string

รูปแบบ Document

object



เพิ่ม Document ด้วยคำสั่ง

```
db.Employees.insertOne(  
  "employee_name": "Edward Bill",  
  "employee_skills": {skill1:"Software Development", skill2:"C++", skill3:"Java"}  
))
```

สอบถามข้อมูล (findOne)

- คำสั่ง findOne ใช้ในการค้นหาเอกสารเพียงเอกสารเดียวจากคอลเลกชันที่ตรงกับเงื่อนไขที่ระบุ หากมีหลายเอกสารที่ตรงเงื่อนไข คำสั่งนี้จะคืนเอกสารแรกที่พบเท่านั้น
- โครงสร้างคำสั่ง: `db.collection_name.findOne(query, projection)`
 - query: เป็นเงื่อนไขที่ใช้ในการค้นหา (เช่น { "name": "John" })
 - projection (ไม่บังคับ): ใช้ในการระบุฟิลด์ที่ต้องการดึงออกมาจากเอกสาร (เช่น { "name": 1, "age": 1 } เพื่อแสดงเฉพาะฟิลด์ name และ age)

เช่น ค้นหา employee_name ที่มีค่า Edward Bill

```
db.Employees.findOne({"employee_name": "Edward Bill"})
```



สอบถามข้อมูล (find)

- คำสั่ง find ใช้ในการค้นหาหลายเอกสารที่ตรงกับเงื่อนไขที่ระบุ โดยจะคืนค่าผลลัพธ์เป็น cursor ซึ่งสามารถวนลูปผ่านเอกสารเหล่านั้นได้ หรือสามารถใช้เมธอด toArray() เพื่อแปลงผลลัพธ์เป็นอาร์เรย์
- โครงสร้างคำสั่ง: db.collection_name.find(query, projection)
 - query: เป็นเงื่อนไขที่ใช้ในการค้นหา (เช่น { "age": { "\$gt": 25 } }) ใน MongoDB \$gt (greater than) เป็นโอเปอเรเตอร์ที่ใช้ในการค้นหาข้อมูลที่มีค่ามากกว่าค่าที่กำหนด
 - projection (ไม่บังคับ): ใช้ในการระบุฟิลด์ที่ต้องการดึงออกมา เช่นเดียวกับ findOne
- เช่น ค้นหา employee_skills ที่มีค่า Java

```
db.Employees.find( { "employee_skills": "Java" } )
```



สอบถามข้อมูลแบบเงื่อนไขเดียว

- การสอบถามข้อมูลตามเงื่อนไขเดียว เช่น การค้นหาเอกสารที่มีค่าของฟิลด์หนึ่งตรงตามที่ต้องการ คุณสามารถใช้คำสั่ง `.find()` กับเงื่อนไขที่ต้องการในรูปแบบ JSON ตัวอย่างเช่น:
- Query Operators:
 - `$eq`: ตรวจสอบความเท่ากัน (equals).
 - `$ne`: ตรวจสอบความไม่เท่ากัน (not equal).
 - `$gt`: ตรวจสอบค่ามากกว่า (greater than).
 - `$gte`: ตรวจสอบค่ามากกว่าหรือเท่ากับ (greater than or equal to).
 - `$lt`: ตรวจสอบค่าน้อยกว่า (less than).
 - `$lte`: ตรวจสอบค่าน้อยกว่าหรือเท่ากับ (less than or equal to).
- เช่น ค้นหาเอกสารในคอลเลกชันที่ฟิลด์ `age` มีค่ามากกว่าหรือเท่ากับ 30.

```
db.collection.find({ "age": { "$gte": 30 } })
```



สอบถามข้อมูลแบบ 2 เงื่อนไข และ หลายเงื่อนไข

- ใช้ \$and ถ้าต้องการให้เอกสารตรงตามเงื่อนไขทั้งสอง (AND):
 - โครงสร้างคำสั่ง: `db.ชื่อcollection.find( { "$and": [ { "field1": value1 }, { "field2": value2 } ] } )`
- ใช้ \$or ถ้าต้องการให้เอกสารตรงตามเงื่อนไขใดเงื่อนไขหนึ่ง (OR):
 - โครงสร้างคำสั่ง: `db.ชื่อcollection.find( { "$or": [ { "field1": value1 }, { "field2": value2 } ] } )`
- ใช้หลายเงื่อนไขในรูปแบบ JSON สามารถรวมเงื่อนไขหลายตัวโดยการใช้คอมม่า (,) ภายใน {} โดยที่ MongoDB จะเข้าใจว่าต้องการให้เอกสารตรงตามเงื่อนไขทั้งหมด (AND):
 - โครงสร้างคำสั่ง: `db.ชื่อcollection.find( { "field1": value1, "field2": value2 , "field3": value3 } )`
- เช่น ค้นหาเอกสารใน Employees ว่าที่ employee_name มีค่า Jojo Lee และ employee_skills มีค่า Java

```
db.Employees.find( { "employee_name": "Jojo Lee", "employee_skills": "Java" } )
```



สอบถามข้อมูลแบบ Object

- ค้นหาเอกสารตามค่าในฟิลด์ย่อยได้โดยการระบุเส้นทางของฟิลด์ย่อยในเงื่อนไขการค้นหา
- สมมุติว่าเอกสารในคอลเลกชันของมีโครงสร้างดังนี้:

```
db.person.insertOne({  
  "_id": 1,  
  "name": "John Doe",  
  "address": {  
    "street": "123 Main St",  
    "city": "New York",  
    "zip": "10001"  
  }  
})
```

เช่น

ค้นหาเอกสารที่ address.city เป็น "New York": `db.person.find({ "address.city": "New York" })`

ค้นหาเอกสารที่ address.city เป็น "New York" และ address.zip เป็น "10001":

```
db.person.find({ "address.city": "New York", "address.zip": "10001" })
```



สอบถามข้อมูลแบบ Array

- ใช้โอเปอเรเตอร์ที่เกี่ยวข้องกับอาร์เรย์ เช่น \$elemMatch, \$in, \$all, และ \$size เพื่อจัดการกับการค้นหาในอาร์เรย์
- ตัวอย่างโครงสร้างเอกสาร: สมมุติว่าเอกสารในคอลเลกชันมีโครงสร้างดังนี้:
 - db.person.insertOne(

```
{
  "_id": 1,
  "name": "John Doe",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001",
    "phoneNumbers": [
      { "type": "home", "number": "555-1234" },
      { "type": "work", "number": "555-5678" }
    ]
  }
}
```

)
 - ค้นหาเอกสารที่มีค่าในฟิลด์ย่อยของอาร์เรย์
 - เช่น ค้นหาเอกสารที่ address.phoneNumbers มีออบเจกต์ที่ number มีค่า "555-1234":

```
db.person.find({
  "address.phoneNumbers.number": "555-1234"
})
```



สอบถามข้อมูลแบบ Array

- ใช้โอเปอเรเตอร์ที่เกี่ยวข้องกับอาร์เรย์ เช่น \$elemMatch, \$in, \$all, และ \$size เพื่อจัดการกับการค้นหาในอาร์เรย์
- ตัวอย่างโครงสร้างเอกสาร: สมมติว่าเอกสารในคอลเลกชันมีโครงสร้างดังนี้:


```
db.person.insertOne({
  "_id": 1,
  "name": "John Doe",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001",
    "phoneNumbers": [
      { "type": "home", "number": "555-1234" },
      { "type": "work", "number": "555-5678" }
    ]
  }
})
```
- \$elemMatch ใช้ในการค้นหาเอกสารที่มีอาร์เรย์ที่มีออบเจกต์ตรงตามเงื่อนไขหลายเงื่อนไขพร้อมกัน
- เช่น ค้นหาเอกสารที่ address.phoneNumbers มีออบเจกต์ที่ type เป็น "home" และ number เป็น "555-1234":


```
db.person.find({
  "address.phoneNumbers": {
    "$elemMatch": { "type": "home", "number": "555-1234" } } })
```



สอบถามข้อมูลแบบ Array

- ใช้โอเปอเรเตอร์ที่เกี่ยวข้องกับอาร์เรย์ เช่น \$elemMatch, \$in, \$all, และ \$size เพื่อจัดการกับการค้นหาในอาร์เรย์
- ตัวอย่างโครงสร้างเอกสาร: สมมติว่าเอกสารในคอลเลกชันมีโครงสร้างดังนี้:


```
db.person.insertOne({
  "_id": 1,
  "name": "John Doe",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001",
    "phoneNumbers": [
      { "type": "home", "number": "555-1234" },
      { "type": "work", "number": "555-5678" }
    ]
  }
})
```
- \$in ใช้ในการค้นหาข้อมูลที่ค่าของฟิลด์ตรงกับค่าใดค่าหนึ่งในอาร์เรย์ที่กำหนด ตัวอย่าง เช่น ถ้าต้องการค้นหาเอกสารที่ฟิลด์หนึ่งมีค่าเป็นหนึ่งในหลายๆ ค่า สามารถใช้ \$in เพื่อระบุค่าเหล่านั้นได้
- เช่น ค้นหาเอกสารที่ address.phoneNumbers.number มีค่าเป็น "555-1234" หรือ "555-5678":


```
db.person.find({
  "address.phoneNumbers.number": {
    "$in": ["555-1234", "555-5678"] } })
```



สอบถามข้อมูลแบบ Array

- ใช้โอเปอเรเตอร์ที่เกี่ยวข้องกับอาร์เรย์ เช่น \$elemMatch, \$in, \$all, และ \$size เพื่อจัดการกับการค้นหาในอาร์เรย์
- ตัวอย่างโครงสร้างเอกสาร: สมมุติว่าเอกสารในคอลเลกชันมีโครงสร้างดังนี้:
 - db.person.insertOne({


```

_id: 1,
"name": "John Doe",
"address": {
  "street": "123 Main St",
  "city": "New York",
  "zip": "10001",
  "phoneNumbers": [
    { "type": "home", "number": "555-1234" },
    { "type": "work", "number": "555-5678" }
  ]
}
})

```
 - \$all ใช้เมื่อต้องการค้นหาเอกสารที่มีอาร์เรย์ที่ประกอบด้วยทุกค่าที่ระบุในอาร์เรย์เงื่อนไข
 - เช่น ค้นหาเอกสารที่ address.phoneNumbers มีทั้ง type เป็น "home" และ type เป็น "work":


```

db.person.find(
  "address.phoneNumbers": {
    "$all": [
      { "$elemMatch": { "type": "home" } },
      { "$elemMatch": { "type": "work" } }
    ]
  }
})

```

สอบถามข้อมูลแบบ Array

- ใช้โอเปอเรเตอร์ที่เกี่ยวข้องกับอาร์เรย์ เช่น \$elemMatch, \$in, \$all, และ \$size เพื่อจัดการกับการค้นหาในอาร์เรย์
- ตัวอย่างโครงสร้างเอกสาร: สมมติว่าเอกสารในคอลเลกชันมีโครงสร้างดังนี้:


```
db.person.insertOne({
  "_id": 1,
  "name": "John Doe",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001",
    "phoneNumbers": [
      { "type": "home", "number": "555-1234" },
      { "type": "work", "number": "555-5678" }
    ]
  }
})
```
- \$size ใช้ในการค้นหาเอกสารตามขนาดของอาร์เรย์ (จำนวนออบเจกต์ในอาร์เรย์)
- เช่น ค้นหาเอกสารที่ address.phoneNumbers มีจำนวนออบเจกต์ทั้งหมด 2 อัน:

```
db.person.find({
  "address.phoneNumbers": { "$size": 2 }
})
```



สร้าง Index

- การสร้าง Index ใน MongoDB มีประโยชน์หลัก ๆ ในการเพิ่มประสิทธิภาพการสืบค้นข้อมูลและปรับปรุงประสิทธิภาพในการจัดการข้อมูลภายในคอลเลกชัน
- Index ช่วยให้ MongoDB ค้นหาเอกสารได้รวดเร็วกว่าการสแกนข้อมูลทั้งหมด (Full Collection Scan) ทุกครั้งที่มีการค้นหา
- การสืบค้นข้อมูลใน MongoDB ที่มีการสร้าง Index จะลดเวลาในการประมวลผลได้มากเมื่อเปรียบเทียบกับคอลเลกชันที่ไม่มี Index
- MongoDB ใช้คำสั่ง `createIndex()` เพื่อสร้าง Index บนคอลเลกชันที่ต้องการ โดยกำหนดฟิลด์ที่ต้องการสร้าง Index และลักษณะของ Index เช่น การเรียงลำดับ
- รูปแบบโครงสร้างคำสั่ง `db.collection.createIndex({ <field1>: <order1>, <field2>: <order2>, ... })`
 - <field> คือฟิลด์ที่ต้องการสร้าง Index
 - <order> คือการเรียงลำดับของ Index (1 = เรียงจากน้อยไปมาก, -1 = เรียงจากมากไปน้อย)



สร้าง Index และค้นหาข้อมูล

- ตัวอย่างโครงสร้างเอกสาร:

```
db.person.insertMany([  
  { "_id": 1, "name": "John", "age": 30, "department": "IT" },  
  { "_id": 2, "name": "Jane", "age": 25, "department": "HR" },  
  { "_id": 3, "name": "Alice", "age": 28, "department": "IT" },  
  { "_id": 4, "name": "Bob", "age": 35, "department": "Finance" }  
])
```

เช่น สร้าง Index ที่คอลเลคชัน person บนฟิลด์ age โดย 1 = เรียงจากน้อยไปมาก เมื่อสร้าง Index แล้ว การ Query ต่อไปนี้จะใช้ Index เพื่อค้นหาข้อมูลพนักงานที่มีอายุ 30:

```
db.person.createIndex({ age: 1 })
```

```
db.person.find({ age: 30 })
```



ตรวจสอบและการลบ Index ในคอลเลกชัน

- ตรวจสอบ Index ในคอลเลกชัน
 - รูปแบบคำสั่ง `db.ชื่อcollection.getIndexes()`
- ลบ Index ในคอลเลกชัน
 - รูปแบบคำสั่ง `db.ชื่อcollection.dropIndex("index_name")`
 - `index_name` คือชื่อ index ดูค่าได้จากการตรวจสอบ Index ในคอลเลกชัน
 - รูปแบบคำสั่งถ้าต้องการลบ index ทั้งหมด `db.ชื่อcollection.dropIndexes()`



การใช้โอเปอเรเตอร์ใน MongoDB ที่ใช้ในการเพิ่มลบข้อมูล

- \$set: ใช้ในการอัปเดตหรือเพิ่มฟิลด์ในเอกสาร
- \$unset: ใช้ในการลบฟิลด์จากเอกสาร
- \$inc: ใช้ในการเพิ่มค่าตัวเลขในฟิลด์
- \$push: ใช้ในการเพิ่มค่าเข้าไปในอาร์เรย์
- \$pull: ใช้ในการลบค่าจากอาร์เรย์



การอัปเดตข้อมูลใน Collection

- คำสั่งอัปเดตใน MongoDB

- `updateOne()`: ใช้ในการอัปเดตเอกสารตัวแรกที่ตรงตามเงื่อนไข โดยไม่คืนค่าเอกสารที่ถูกอัปเดต
- `findOneAndUpdate()` ใช้ในการอัปเดตเอกสารตัวแรกที่ตรงตามเงื่อนไข โดยคืนค่าเอกสารที่ถูกอัปเดต
- `updateMany()`: ใช้ในการอัปเดตหลายเอกสารที่ตรงตามเงื่อนไข
- `replaceOne()`: ใช้ในการแทนที่เอกสารทั้งตัวที่ตรงตามเงื่อนไข

- โครงสร้างคำสั่ง:

`db.ชื่อcollection.updateOne( { ฟิลต์: 'ค่าของฟิลต์' }, { $set: { ฟิลต์ที่จะแก้: ข้อมูลที่แก้ } })`

เงื่อนไขในการค้นหา

การอัปเดตฟิลต์หรือข้อมูล

- เช่น อัปเดตฟิลต์ `age` ของเอกสารที่ `name` เท่ากับ "John Doe" ให้เป็น 30

```
db.person.updateOne( { name: "John Doe" }, { $set: { age: 30 } })
```

- เช่น อัปเดตฟิลต์ `status` ของเอกสารทั้งหมดที่ `age` มากกว่า 25 ให้เป็น "active"

```
db. person.updateMany( { age: { $gt: 25 } }, { $set: { status: "active" } })
```

ตัวอย่างโครงสร้างเอกสาร:

```
db.person.insertOne({
  "_id": 1, "name": "John Doe",
  "age": 20,
  "address": {"street": "123 Main St",
    "city": "New York", "zip": "10001"},
  "skills": ["JavaScript"],
  "status": "inactive"
})
```



การอัปเดตข้อมูลใน Collection (ต่อ)

- เช่น แทนที่เอกสารที่ name เท่ากับ "Jane Doe" ด้วยเอกสารใหม่ทั้งหมด

```
db.person.replaceOne( { name: "Jane Doe" }, { name: "Jane Doe", age: 28, status: "active" } )
```

- เช่น เพิ่มค่า age ขึ้น 1 และเพิ่ม "MongoDB" เข้าไปในฟิลด์ skills

```
db.person.updateOne(  
  { name: "John Doe" },  
  { $inc: { age: 1 },  
    $push: { skills: "MongoDB" } } )
```

- เช่น ลบฟิลด์ age จากเอกสารที่มี name เป็น "John Doe"

```
db.person.updateOne( { name: "John Doe" }, { $unset: { age: "" } } )
```

- เช่น ลบค่า "JavaScript" จากอาร์เรย์ skills ในเอกสารที่มี name เป็น "John Doe"

```
db.person.updateOne( { name: "John Doe" }, { $pull: { skills: "JavaScript" } } )
```



การลบข้อมูลใน Collection

- deleteOne(): ใช้ในการลบเอกสารแรกตรงตามเงื่อนไข
 - โครงสร้างคำสั่ง: `db.ชื่อcollection.deleteOne({ฟิลด์: 'ค่าของฟิลด์'})`
 เงื่อนไขในการค้นหา
- deleteMany(): ใช้ในการลบเอกสารทั้งหมดตรงตามเงื่อนไข
- findOneAndDelete(): ลบเอกสารแรกตรงตามเงื่อนไขและคืนค่าเอกสารที่ถูกลบ
- เช่น การลบเฉพาะเอกสารของ Jane Doe ที่มี status มีค่า inactive
`db.person.deleteOne({ name: "Jane Doe", status: "inactive" })`
- เช่น การลบทุกเอกสารที่มี status เป็น "inactive":
`db.person.deleteMany({ status: "inactive" })`

ตัวอย่างโครงสร้างเอกสาร:

```
db.person.insertMany([
  { "_id": 1, "name": "John Doe", "status": "active" },
  { "_id": 2, "name": "Jane Doe", "status": "inactive" },
  { "_id": 3, "name": "Edward Bill", "status": "inactive" }
])
```

